AI VIET NAM
@aivietnam.edu.vn

Knowledge Distillation

Extra – Part 1

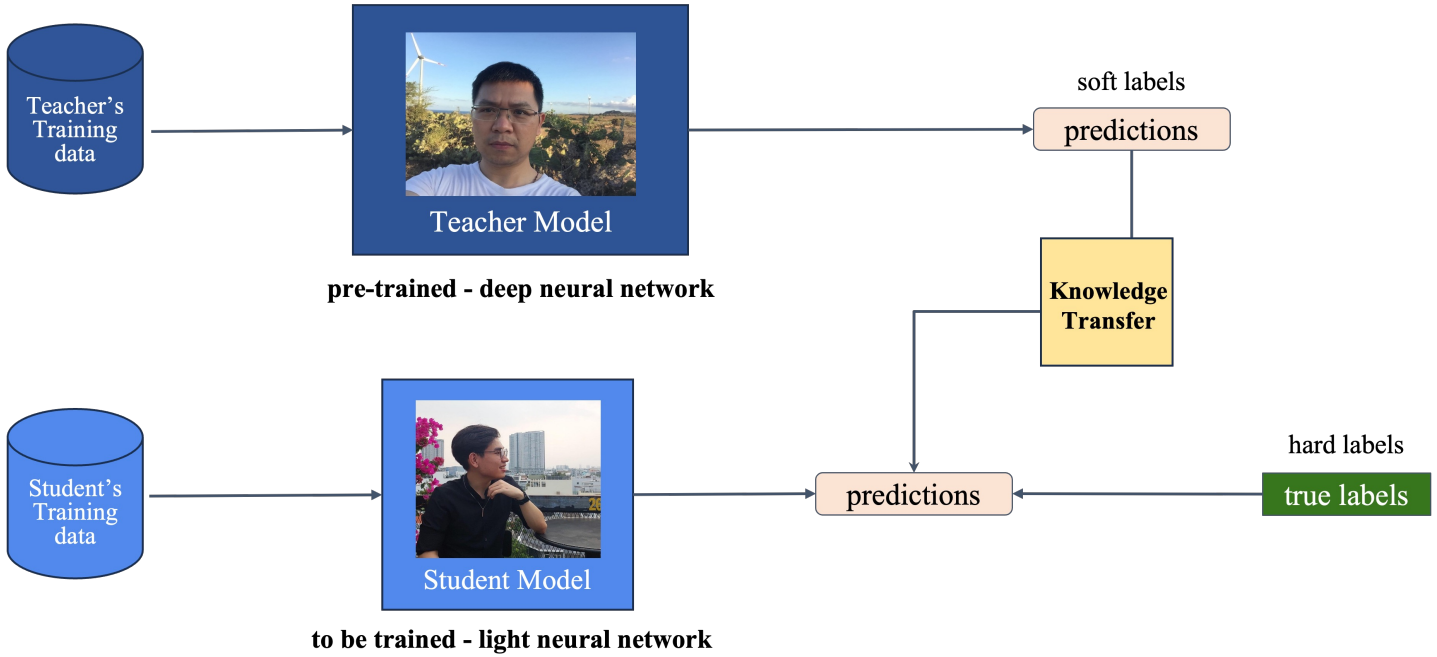
Dinh-Thang Duong – TA

Yen-Linh Vu – STA

Anh-Khoi Nguyen – STA

Getting Started

❖ Content



Our objectives:

- Discuss about the motivation and definition of Knowledge Distillation technique.
- Some current applications of Knowledge Distillation.
- Basic approaches for Knowledge Distillation.
- Investigate how to implement the Response-based Knowledge Distillation method using PyTorch.

AI VIET NAM
@aivietnam.edu.vn

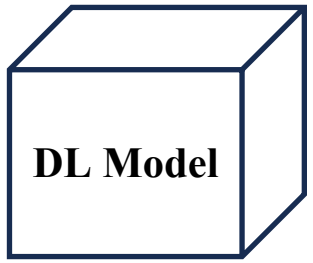
Outline

- Introduction
- Knowledge Distillation
- Response-based Knowledge
- Question

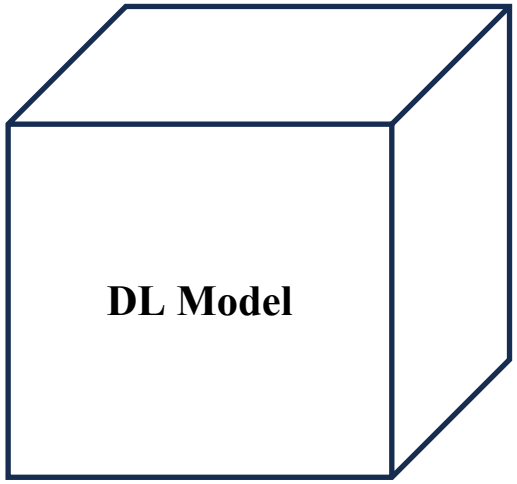
Introduction

Introduction

❖ Getting Started



Small Deep Learning (DL) model.



Large Deep Learning (DL) model.

In Deep Learning, there are models differ in size. In theory, the larger the model, the better the performance.

Introduction

❖ Challenges & Motivation in Deploying Large Language Models (LLMs)

- **LLMs are becoming increasingly popular**
  - For example: GPT-3 has **175B parameters** and large ensemble models.
    - Computationally heavy.
    - resource-intensive.
    - hard to deploy in real-world applications.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table: GPT-3 Model Sizes and Parameters. Source: link.

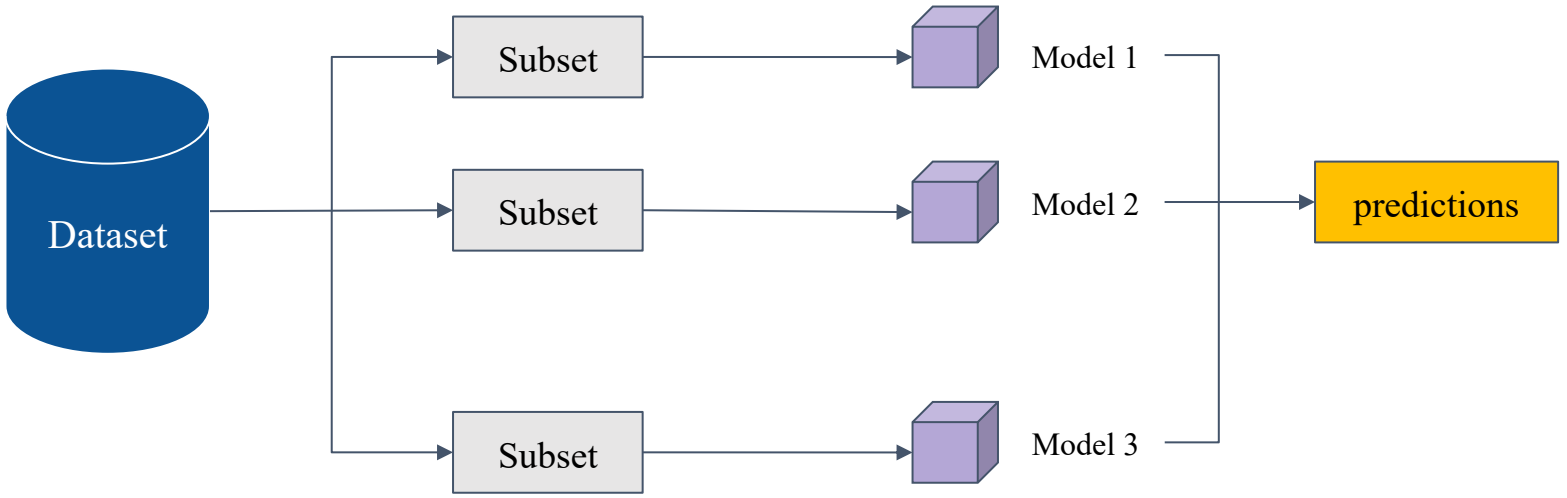


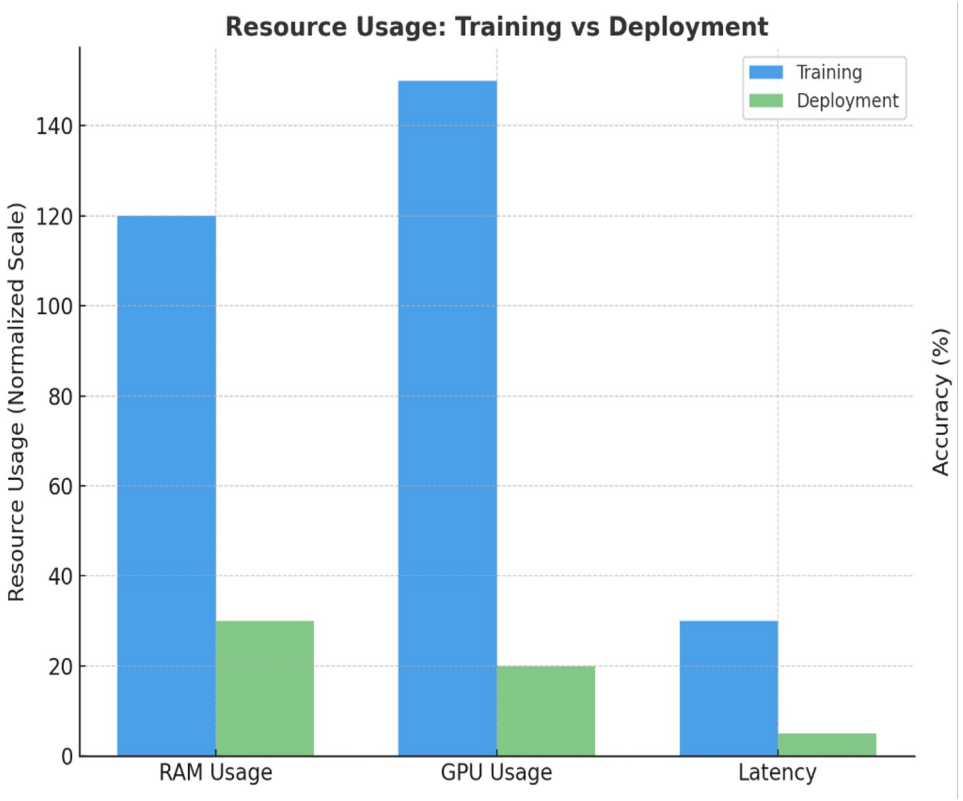
Figure: Ensemble Models Illustration.

Introduction

❖ Challenges & Motivation in Deploying Large Language Models (LLMs)

- **Conflict between training and deployment stages**
  - **Training:** Requires large resources (RAM, GPU), no real-time constraints.
  - **Deployment:** Demands high-speed processing and resource efficiency.

? How can we maintain high performance in large models while ensuring practical deployment?



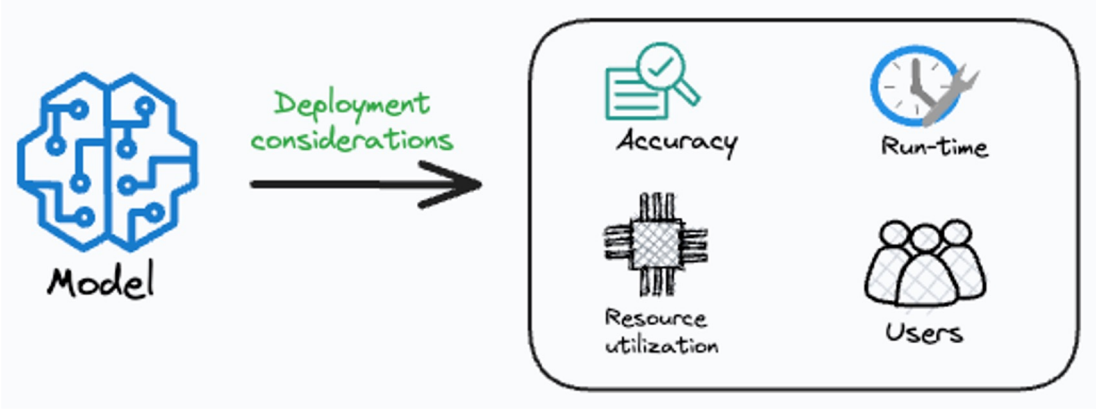


Figure: Deployment requirements. Source: link.

Introduction

❖ Model Compression – A Solution to Simplify Neural Networks

- **Objective:** Reduce the complexity of neural networks while maintaining high accuracy.
- **Main Techniques:**
  - Quantization.
  - Weights Pruning.
  - Knowledge Distillation.
  - High-performance libraries.

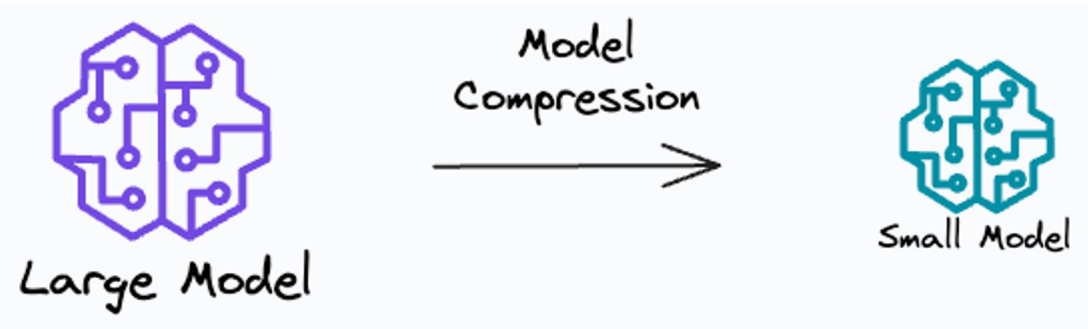
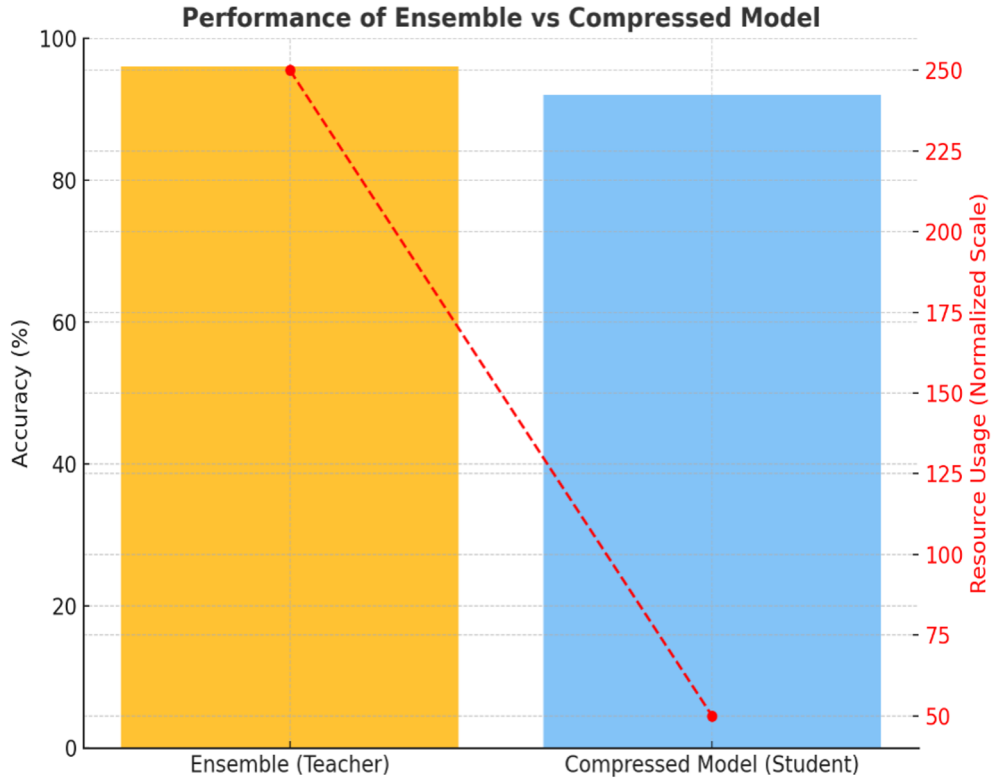


Figure: Model compression. Source: link.



Introduction

❖ Learning from Nature: A Simple Machine Learning Approach

- **Biological Example:**

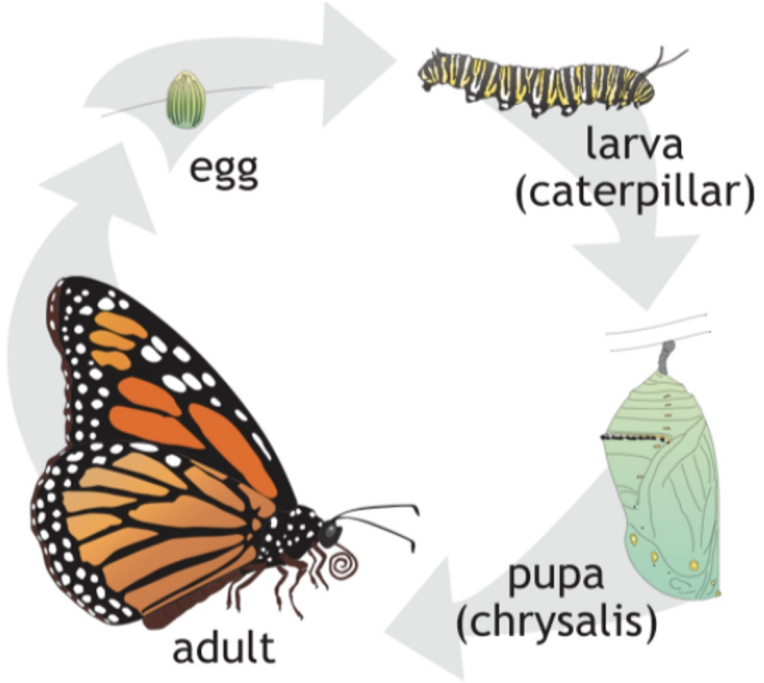


Figure: Butterfly Life Cycle. Source: link.

- **Knowledge Distillation Method:**

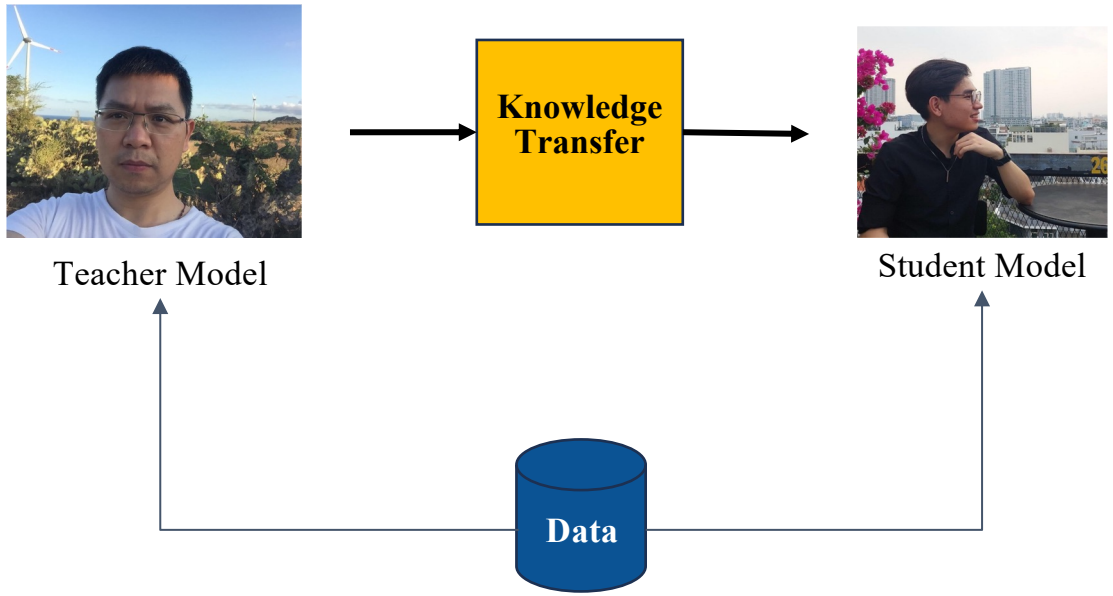
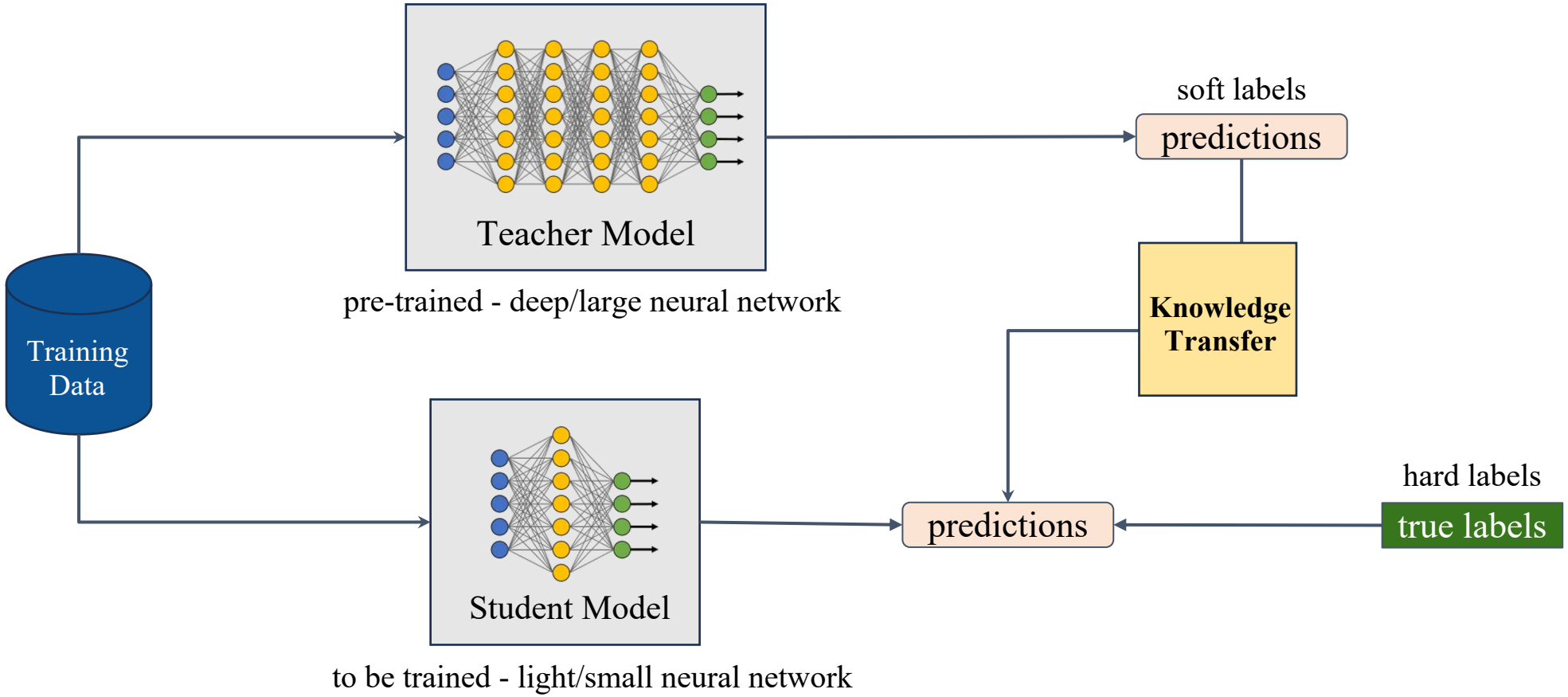


Figure: Simple Visualization of Knowledge Distillation Method.

Introduction

❖ Introduction to Knowledge Distillation

Knowledge Distillation is a technique that allows transferring knowledge from large, computationally expensive models to smaller models without losing effectiveness.



Introduction

❖Survey: Knowledge Distillation Applications

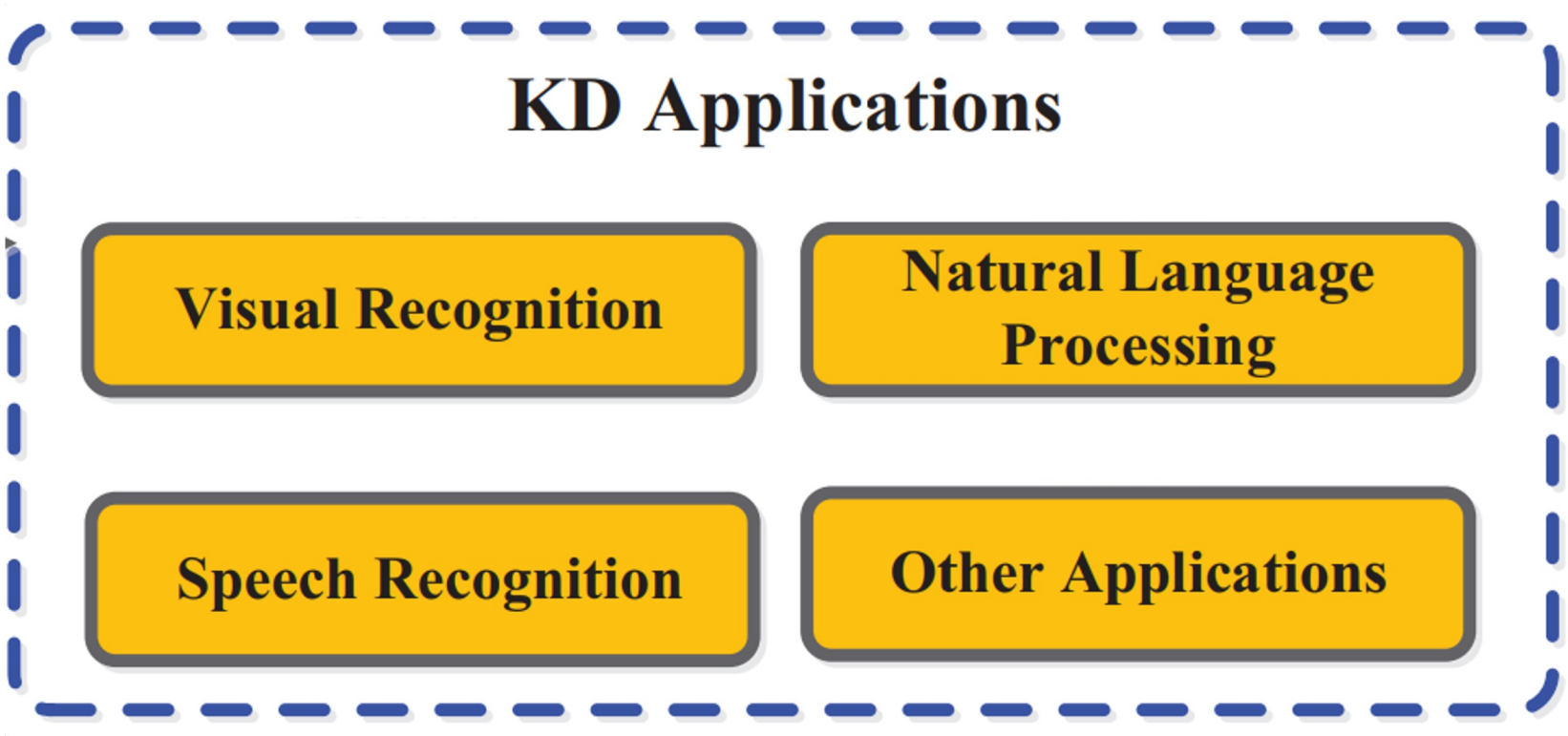


Figure: The applications of knowledge distillation. Source: link.

Introduction

❖Knowledge Distillation: A Survey

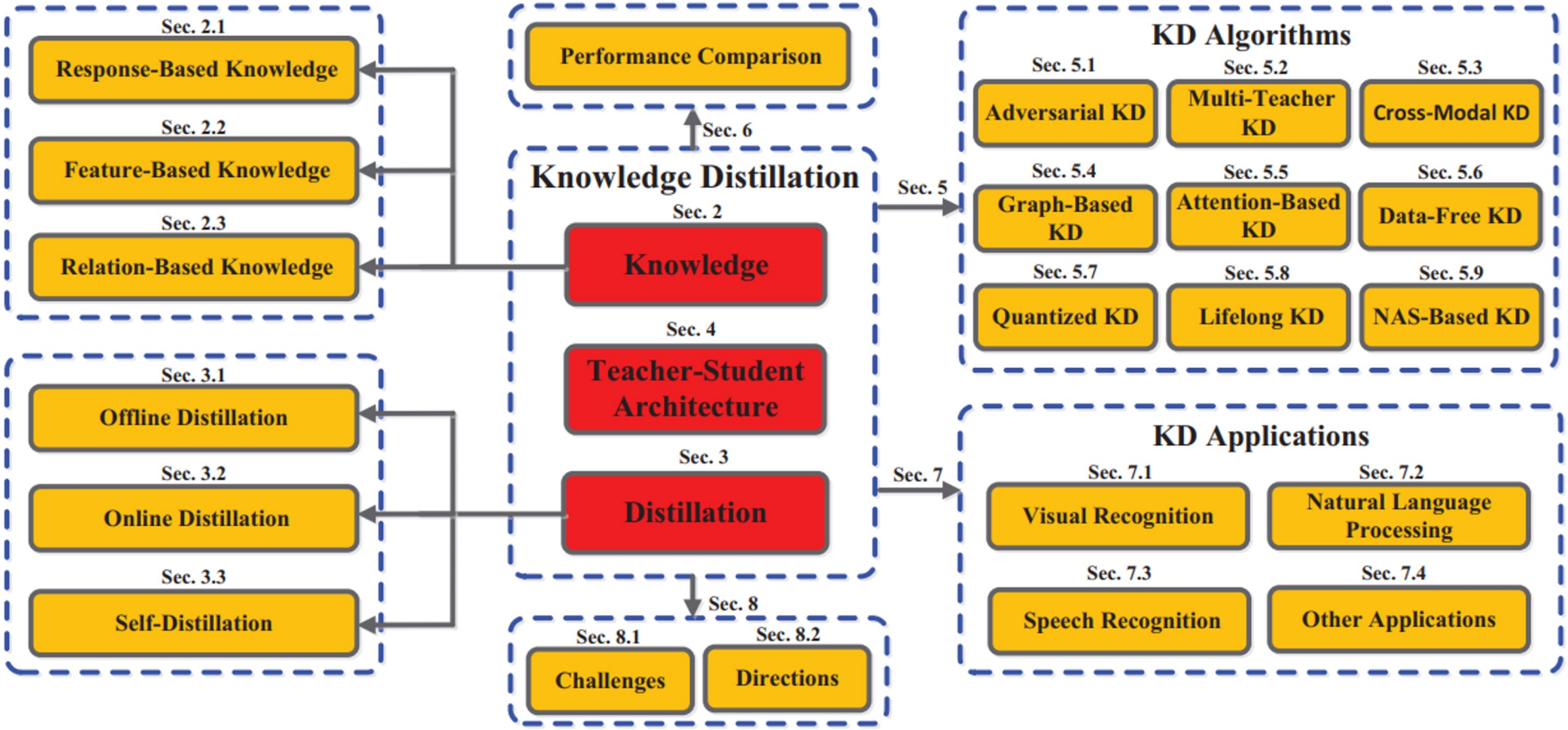


Figure: The schematic structure of knowledge distillation. Source: link.

Introduction

❖ Survey: Knowledge Distillation Research (2015–2024)

- **2015** — **Hinton et al. - Distilling the Knowledge in a Neural Network.**
- **2016** — **Kim et al. - Sequence-Level Knowledge Distillation.**
- **2019** — **Jiao et al. - TinyBERT: Distilling BERT for Natural Language Understanding.**
- **2021** — **Anderson et al. - Compressing Visual-linguistic Model via Knowledge Distillation.**
- **2023** — **Gu et al. - MiniLLM: Knowledge Distillation of Large Language Models.**
- **2024** — **Ko et al. - DistiLLM: Towards Streamlined Distillation for Large Language Models.**

Introduction

❖ Survey: Knowledge Distillation Research (2015–2024)

2015

Hinton et al. - Distilling the Knowledge in a Neural Network.

First introduced the concept of Knowledge Distillation.

Established KD as a method to transfer knowledge from a large "teacher" model to a smaller "student" model via soft logits.

2016

2019

2021

2023

2024

System	Test Frame Accuracy	WER
Baseline	58.9%	10.9%
10xEnsemble	61.1%	10.7%
Distilled Single model	60.8%	10.7%

Table: Frame classification accuracy and WER showing that the distilled single model performs about as well as the averaged predictions of 10 models that were used to create the soft targets. Source: link.

Introduction

❖Survey: Knowledge Distillation Research (2015–2024)

2016

Kim et al. - Sequence-Level Knowledge Distillation.

Pioneered sequence-level KD for natural language processing tasks.

Extended KD to sequence-to-sequence learning, improving tasks like neural machine translation through token-level and sequence-level distillation.

2019

2021

2023

2024

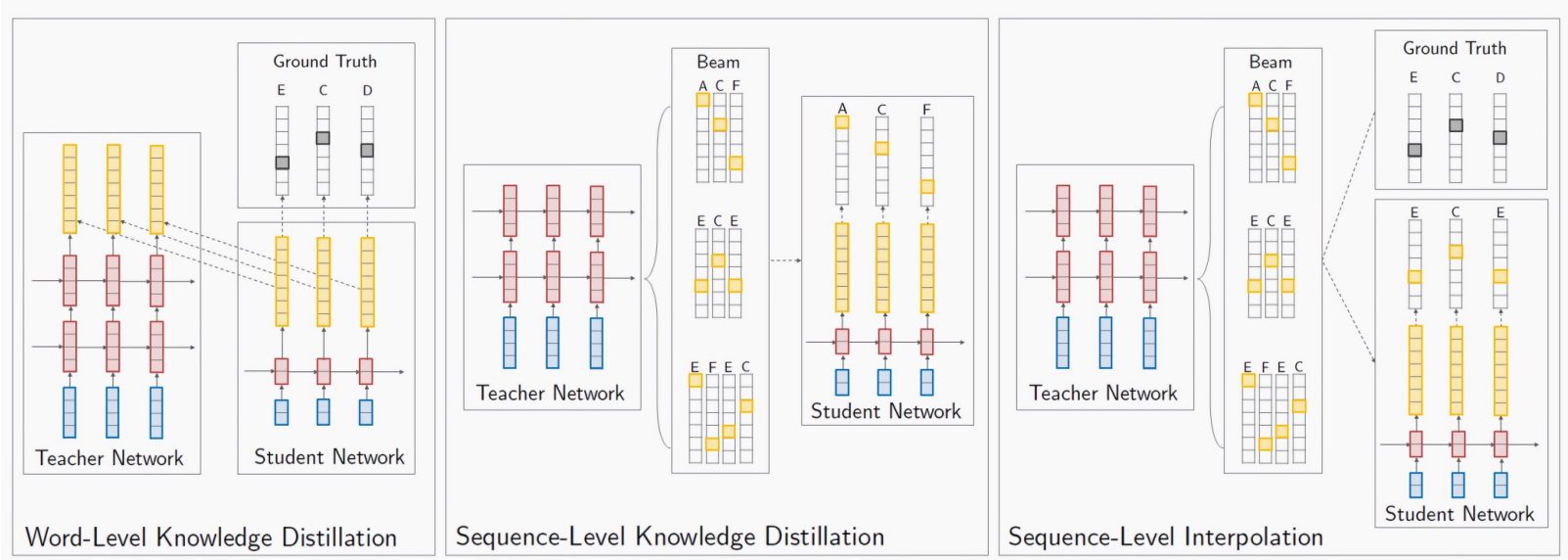


Figure: Overview of the different knowledge distillation approaches. Source: link.

Introduction

❖Survey: Knowledge Distillation Research (2015–2024)

2019

Jiao et al. - TinyBERT: Distilling BERT for Natural Language Understanding.
Contribution to NLP applications applied KD.

First to use layer-wise distillation to create a compact version of BERT, enabling NLP applications on low-resource devices.

2021

2023

2024

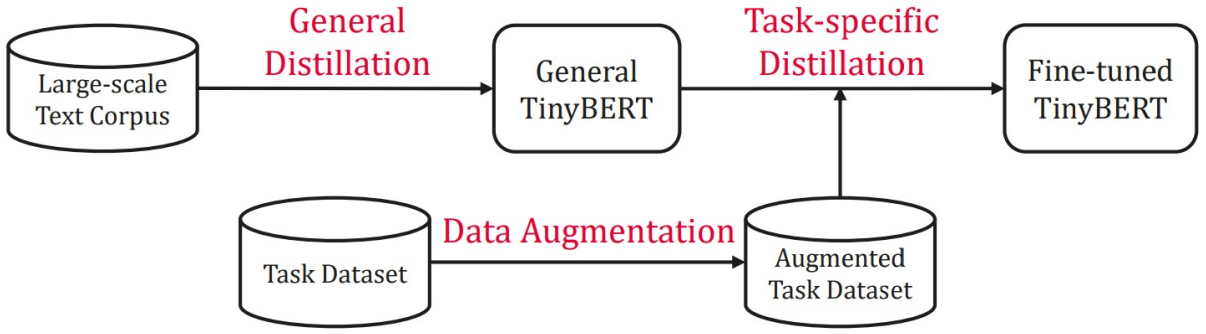


Figure: The illustration of TinyBERT learning. Source: link.



Figure: TinyBERT application visualization.

Introduction

❖Survey: Knowledge Distillation Research (2015–2024)

2021

Anderson et al. - Compressing Visual-linguistic Model via Knowledge Distillation.
First to apply KD to visual-linguistic models.

2023

Adapted KD to multimodal tasks such as Visual Question Answering (VQA) and image captioning, improving efficiency and scalability.

2024

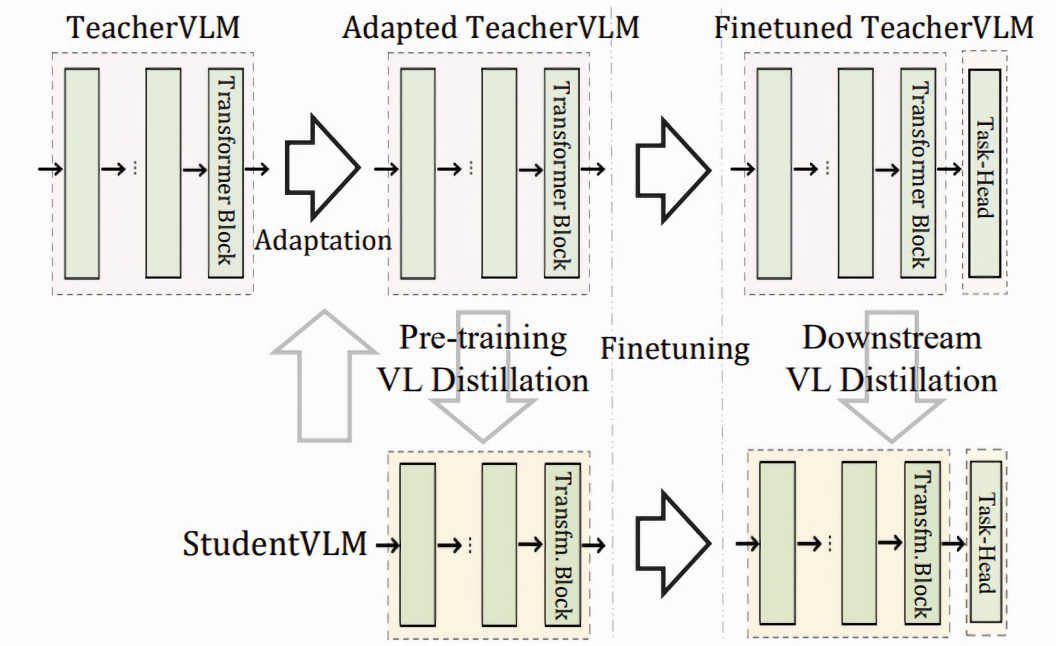


Figure: Overview of proposed VL distillation schema. Source: link.

Introduction

❖Survey: Knowledge Distillation Research (2015–2024)

2023

Gu et al. - MiniLLM: Knowledge Distillation of Large Language Models.
MiniLLM enhances SeqKD by improving efficiency and addressing data sparsity for LLMs.

2024

Applied KD to compress large language models (LLMs) while maintaining high performance, facilitating deployment on edge devices.

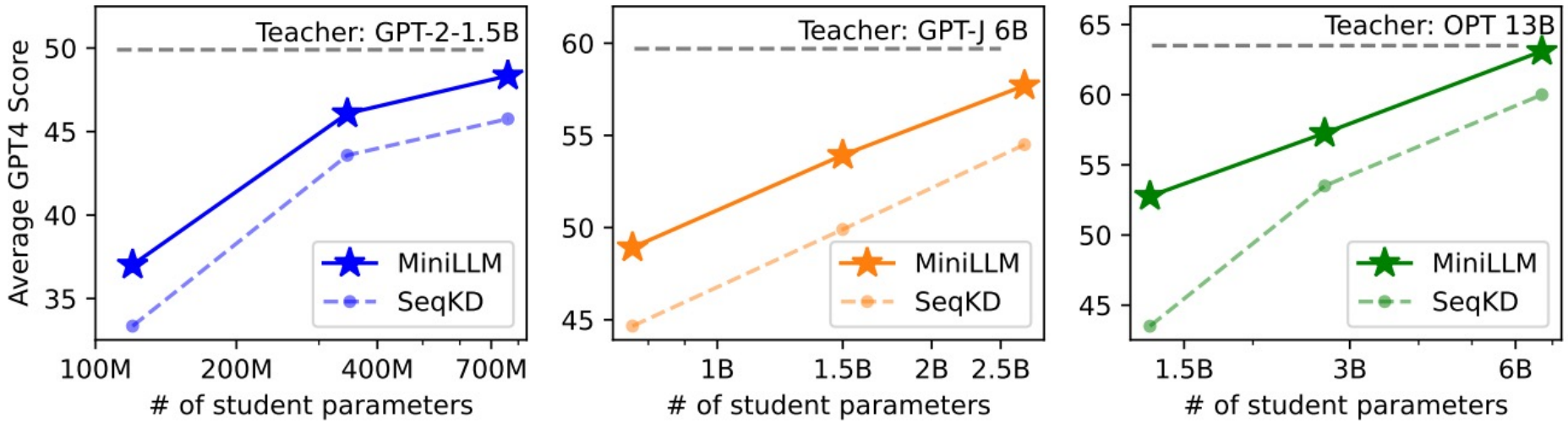


Figure: The comparison of MINILLM with the sequence-level KD (SeqKD).
Source: link.

Introduction

❖Survey: Knowledge Distillation Research (2015–2024)

2024

Ko et al. - DistiLLM: Towards Streamlined Distillation for Large Language Models.
First to refine KD pipelines for real-time LLM applications.

Introduced adaptive techniques to optimize inference and memory efficiency, enabling virtual assistants and real-time NLP systems.

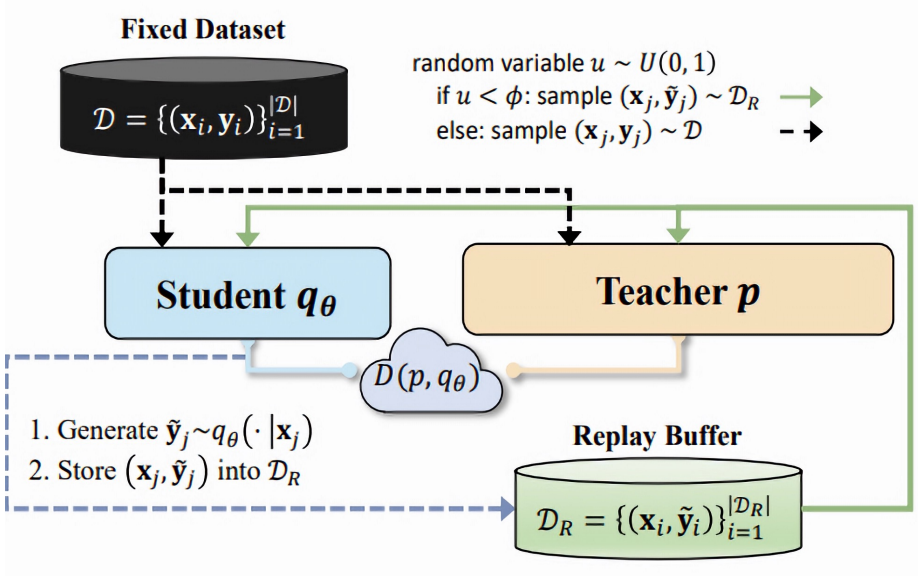


Figure: Adaptive Off-Policy Approaches. Source: link.



Figure: DistiLLM applications visualization.

Knowledge Distillation

Knowledge Distillation

❖ Introduction

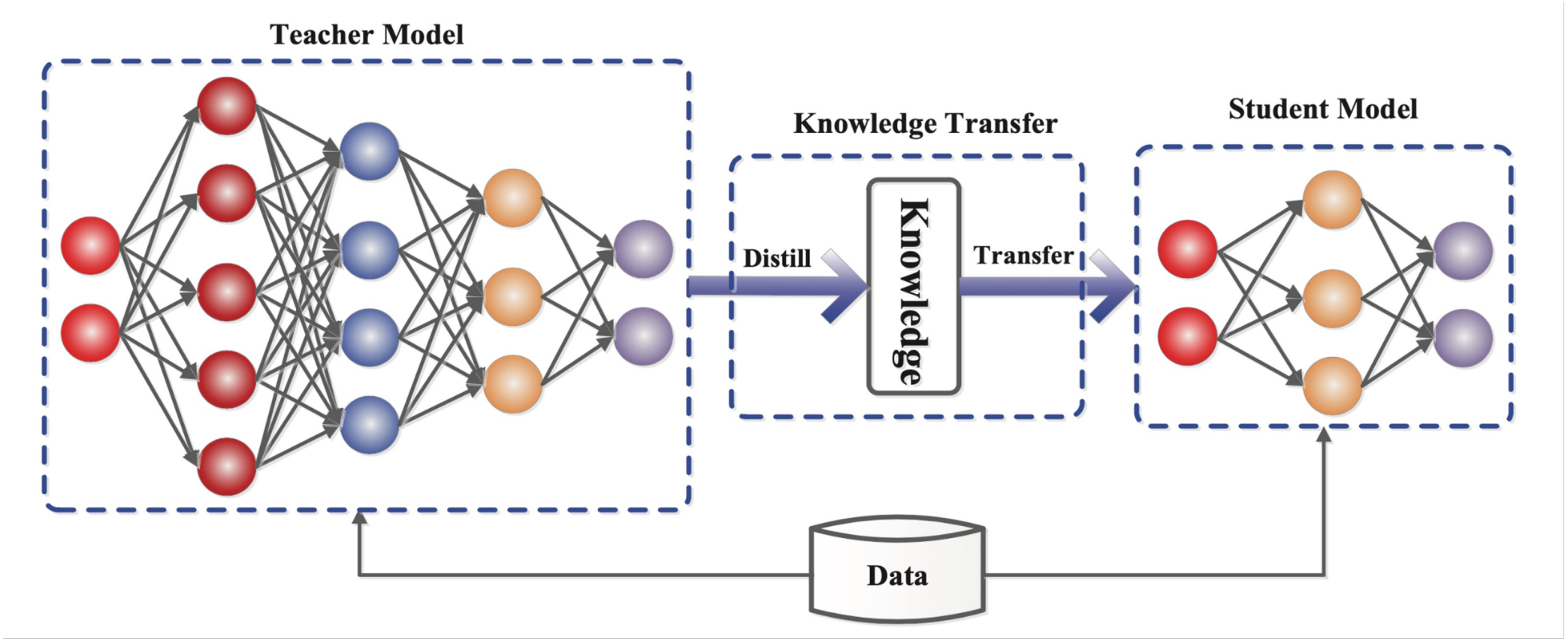


Figure: The generic teacher-student framework for knowledge distillation. Source: link.

Knowledge Distillation

❖ Introduction

KD uses different types of knowledge:

- **Response-based:** Model outputs.
- **Feature-based:** Intermediate layers.
- **Relation-based:** Relationships between data points.

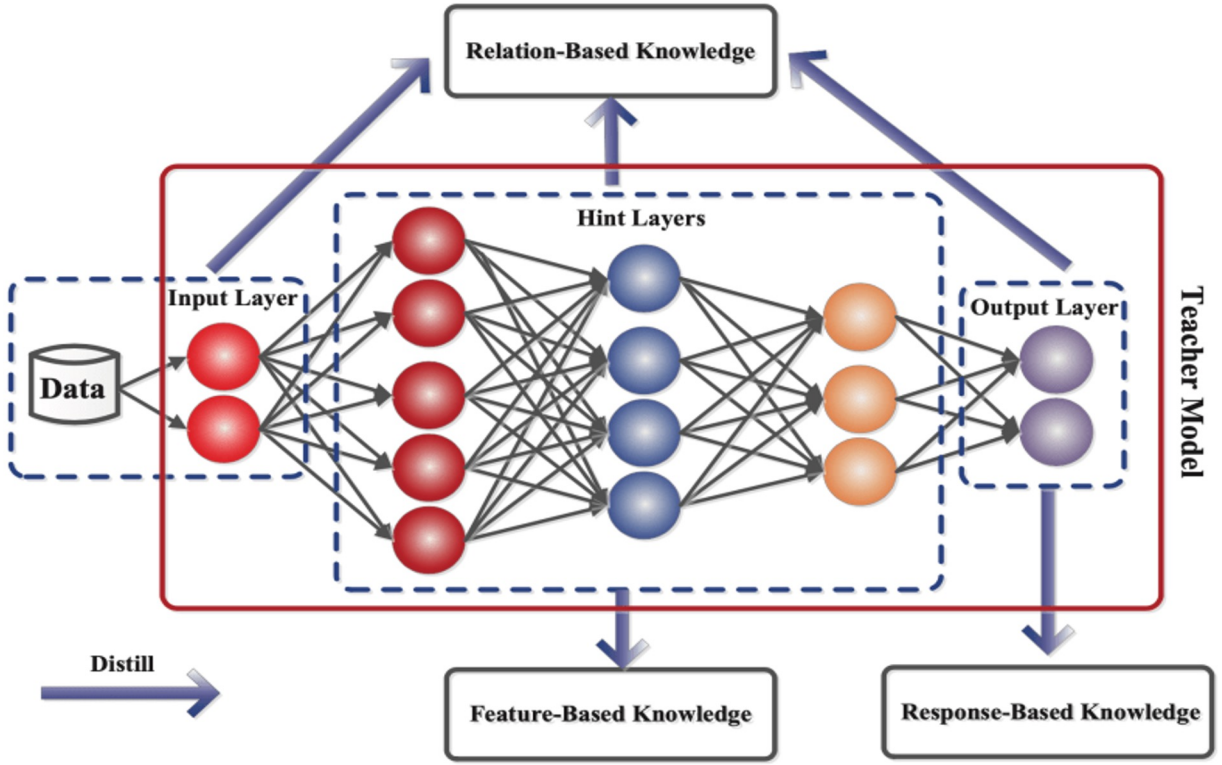


Figure: The schematic illustrations of sources of response-based knowledge, feature-based knowledge and relation-based knowledge in a deep teacher network.. Source: link.

Knowledge Distillation

❖ Introduction

KD uses different types of knowledge:

- **Response-based:** Model outputs.

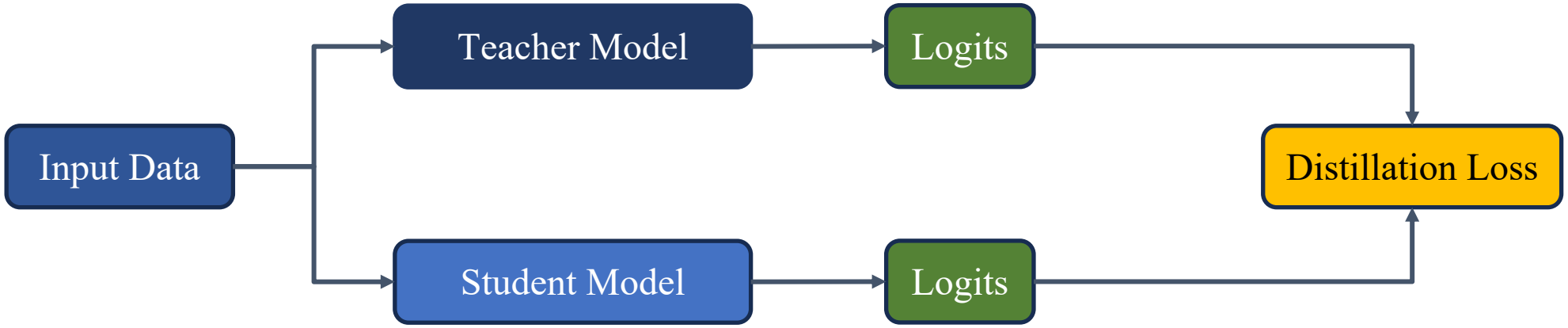
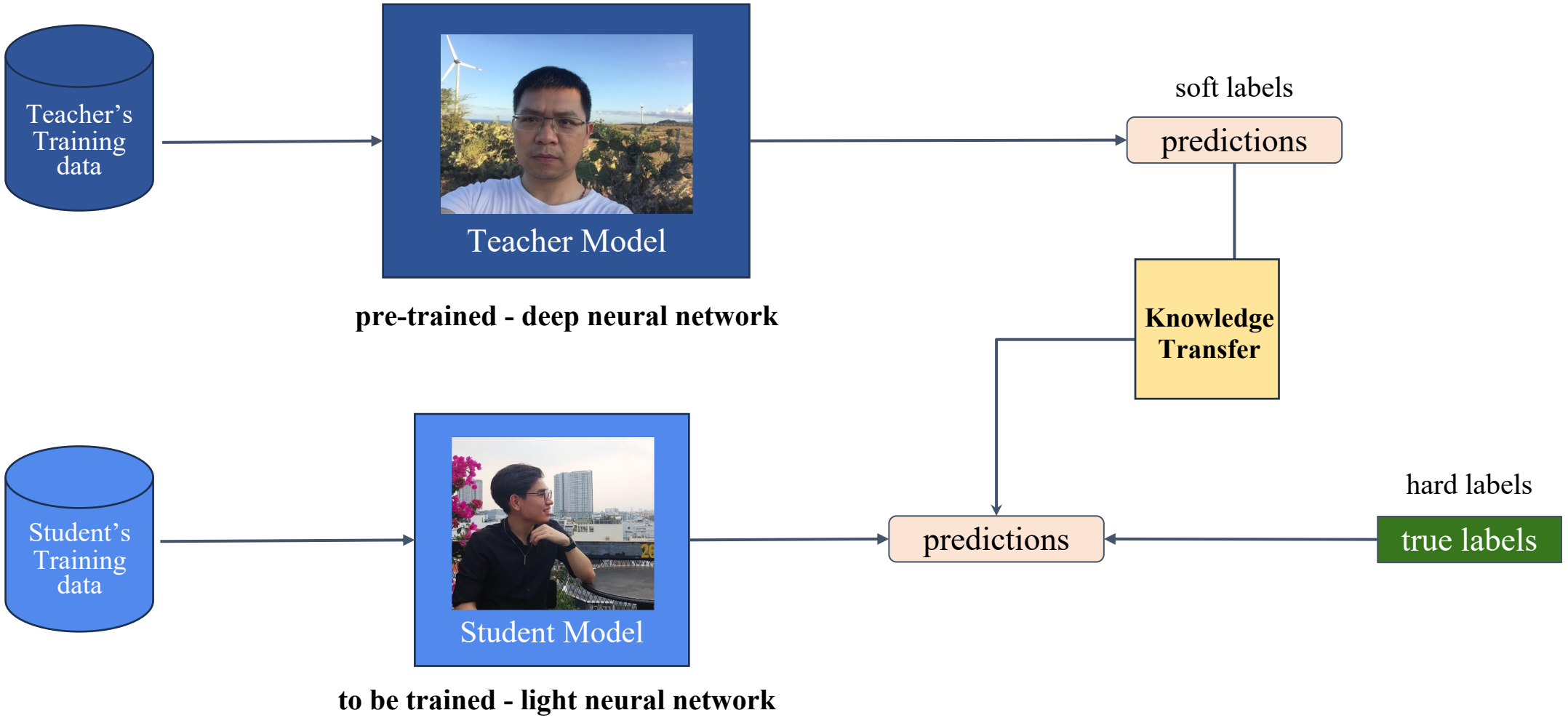


Figure: Response-based Knowledge.

Knowledge Distillation

❖ Introduction to Knowledge Distillation



Knowledge Distillation

❖ Introduction

KD uses different types of knowledge:

- **Feature-based:** Intermediate layers.

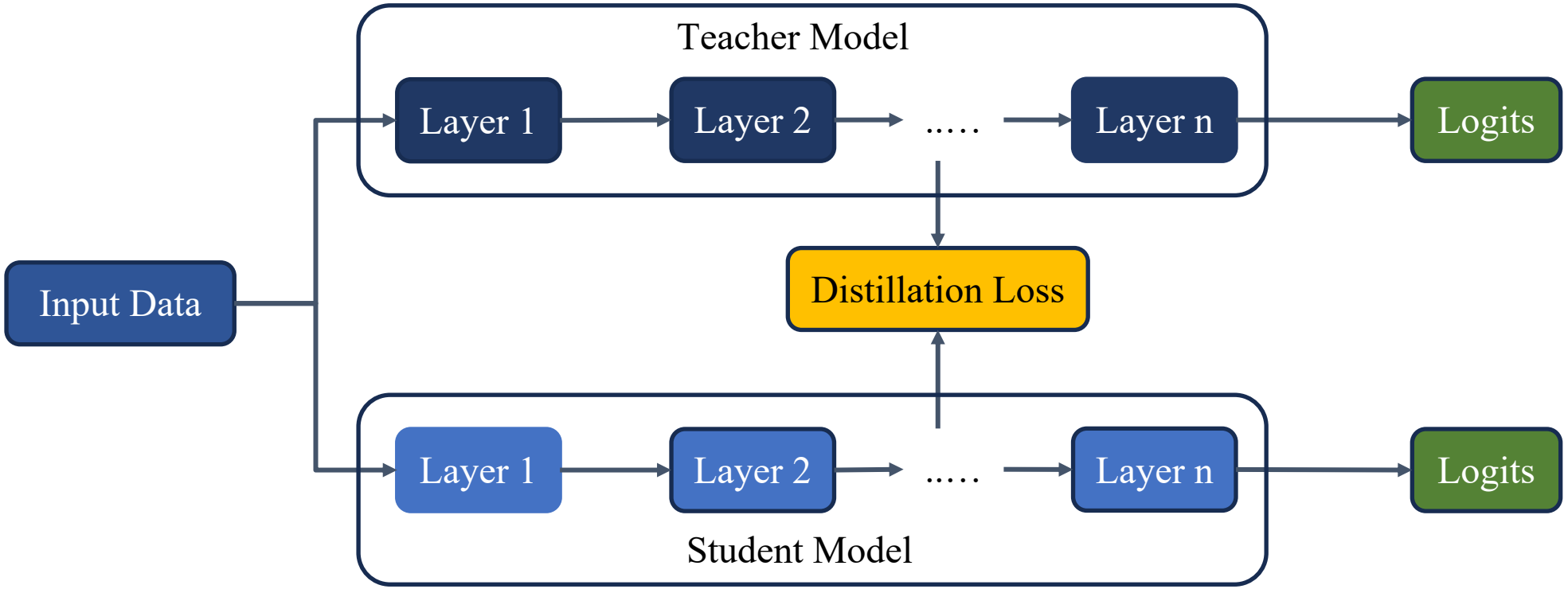


Figure: Feature-based Knowledge.

Knowledge Distillation

❖ Introduction

KD uses different types of knowledge:

- **Relation-based:** Relationships between data points.

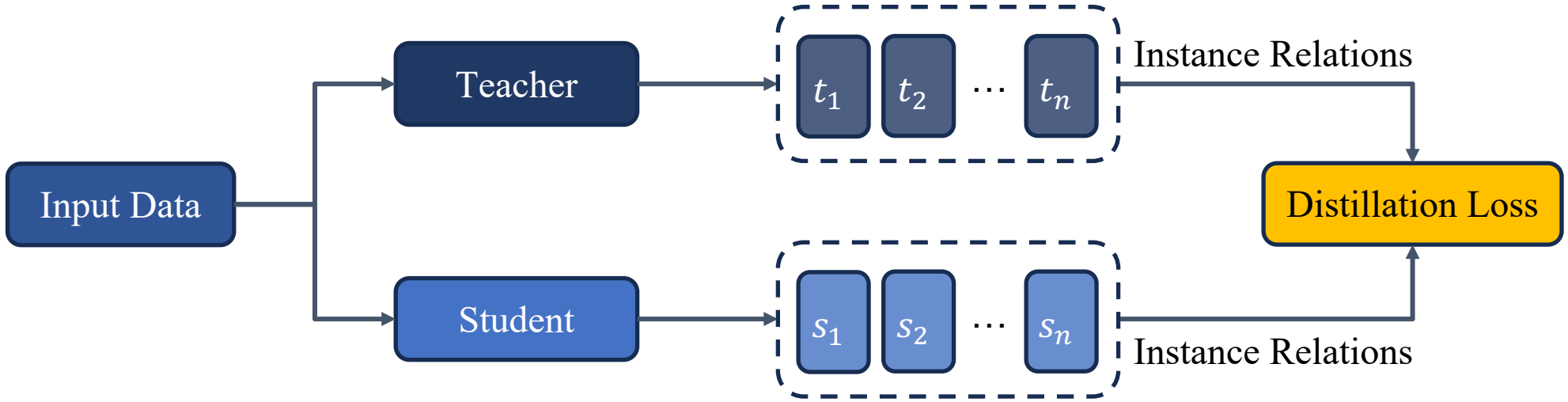


Figure: Relation-based Knowledge.

Response-based Knowledge

Response-based Knowledge

❖ Introduction

Response-based Knowledge Distillation (KD): Utilizes the outputs (logits) of the Teacher model to train the Student model. This is the most common and straightforward method in KD.

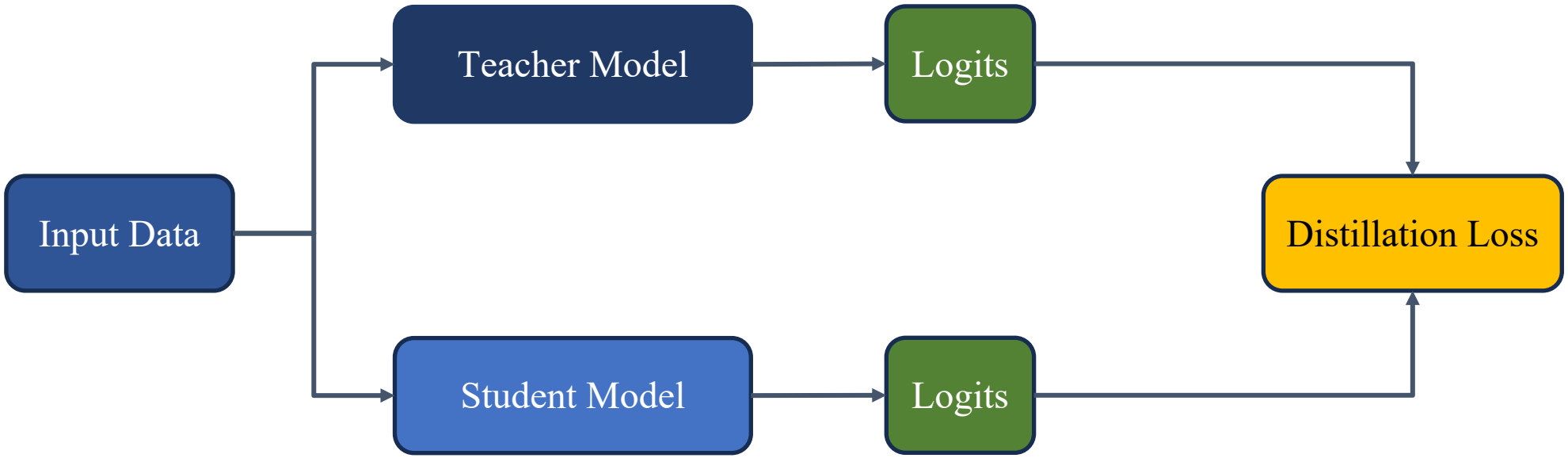
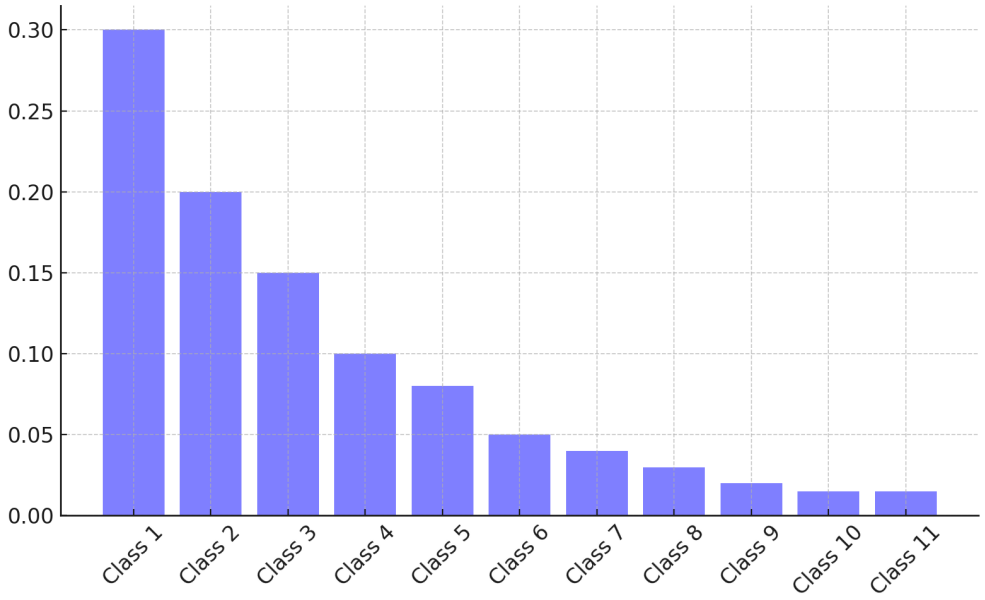


Figure: Response-based Knowledge.

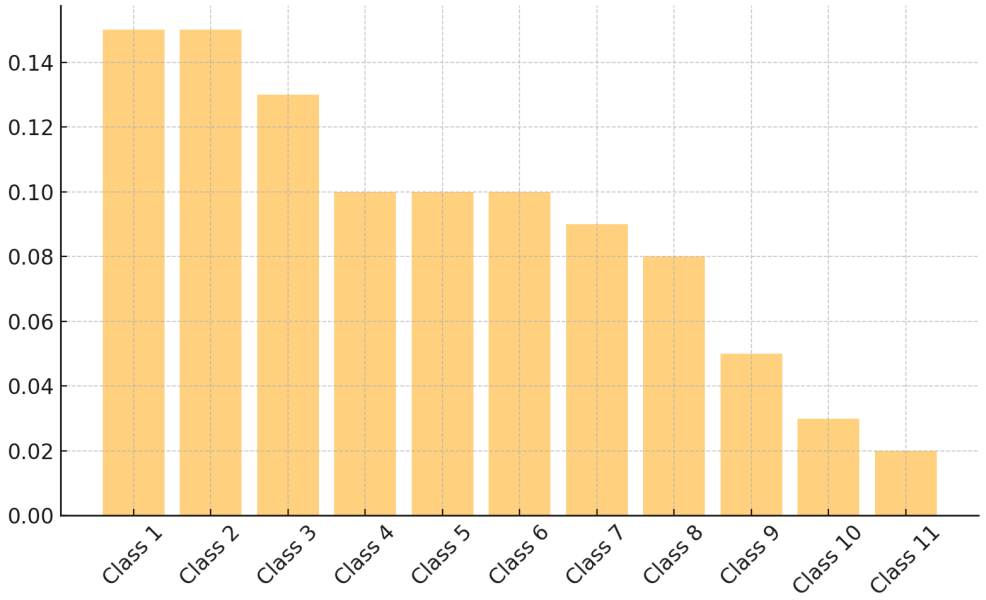
AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Problem: How to distill the knowledge



Teacher Output.



Student Output.

Given knowledge from teacher, how can we transfer it into the student network?

Response-based Knowledge

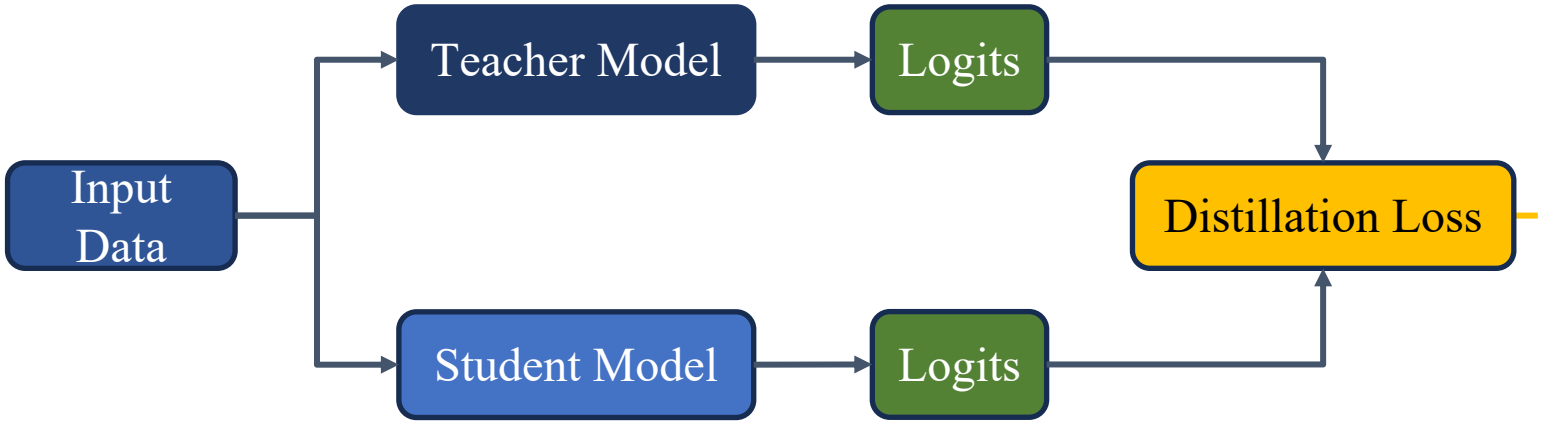
❖ Distillation Loss

- z_t : Teacher model logits.
- z_s : Student model logits.
- $\mathcal{L}_R(z_t, z_s)$: Divergence loss.

where

$$L_{ResD}(z_t, z_s) = \mathcal{L}_R(z_t, z_s)$$

formula

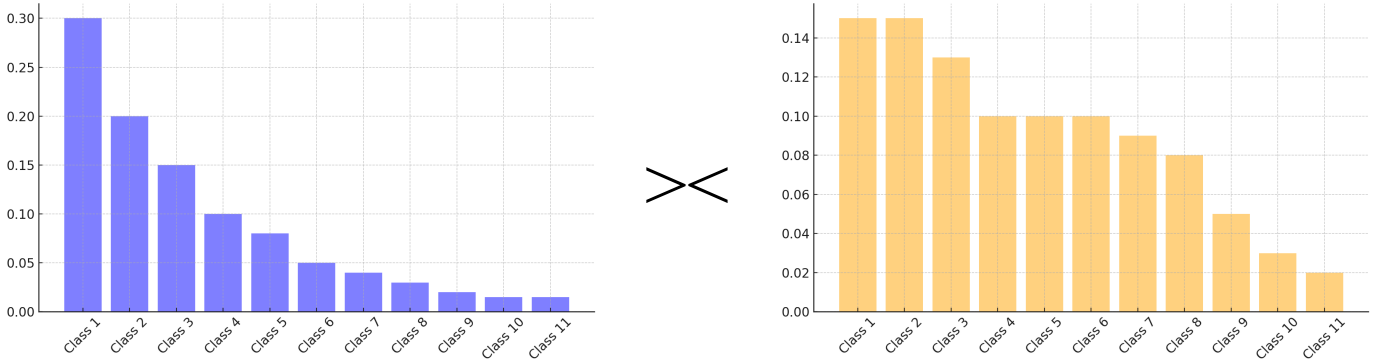


Idea: Let the student try to align its logits with the teacher's logits through loss function.

Figure: The generic response-based knowledge distillation. Source: link.

Response-based Knowledge

❖ Distillation loss: KL-Divergence



Kullback-Leibler Divergence (KL-Divergence): A tool from information theory that measures how one probability distribution differs from another. In the case of this knowledge distillation approach, it helps the student model align its predictions with the teacher's knowledge.

How to mearsure the difference between two distrubutions?

Response-based Knowledge

❖ Distillation loss: KL-Divergence

- Following wikipedia, the KL-Divergence is define as:

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log(\frac{P(x)}{Q(x)})$$

In the case of KD:

- $P_{teacher,i}$: The teacher probability for class i.
- $P_{student,i}$: The student probability for class i.

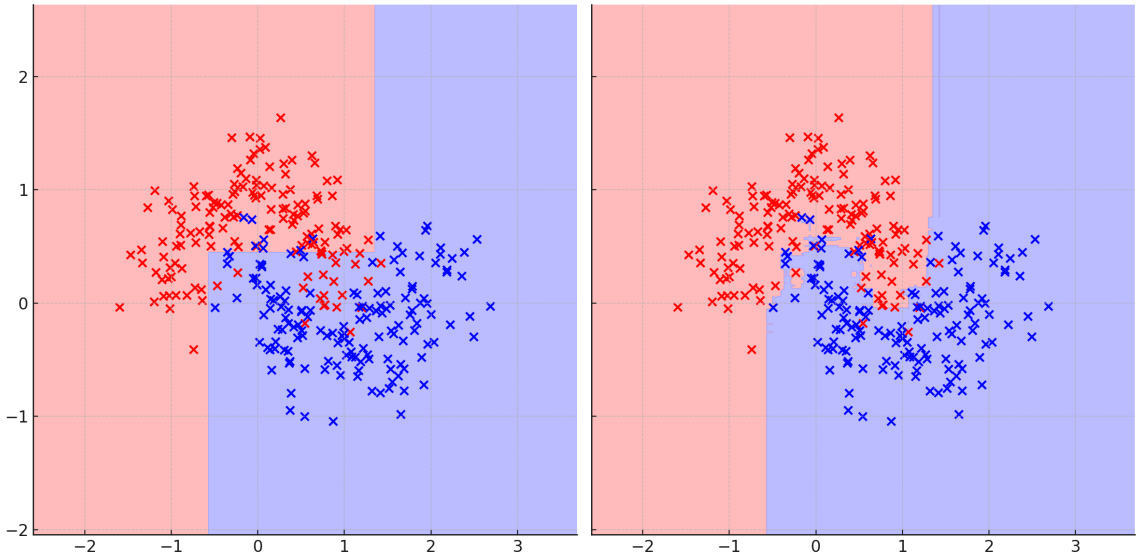
- When applying to the context of Knowledge Distillation (KD), this formula can be written as:

$$D_{KL}(P_{teacher}||P_{student}) = \sum_{i} P_{teacher,i} \cdot (\log P_{teacher,i} - \log P_{student,i})$$

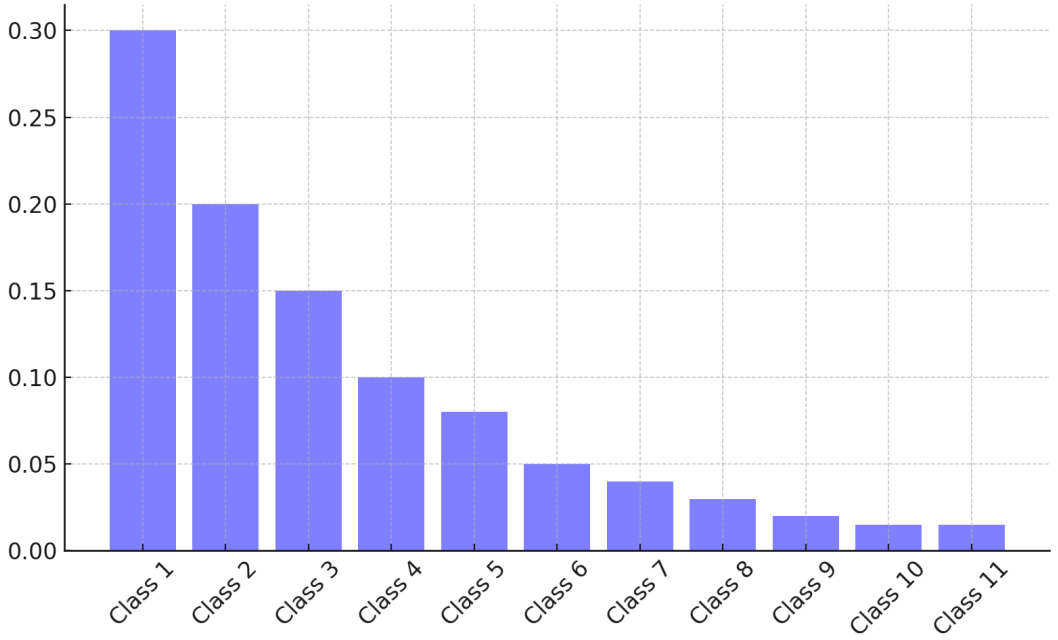
AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ How to distill the knowledge: Generalization



When training a machine learning model, it is often exptected to have a well-generalized model that can work well on unseen data.



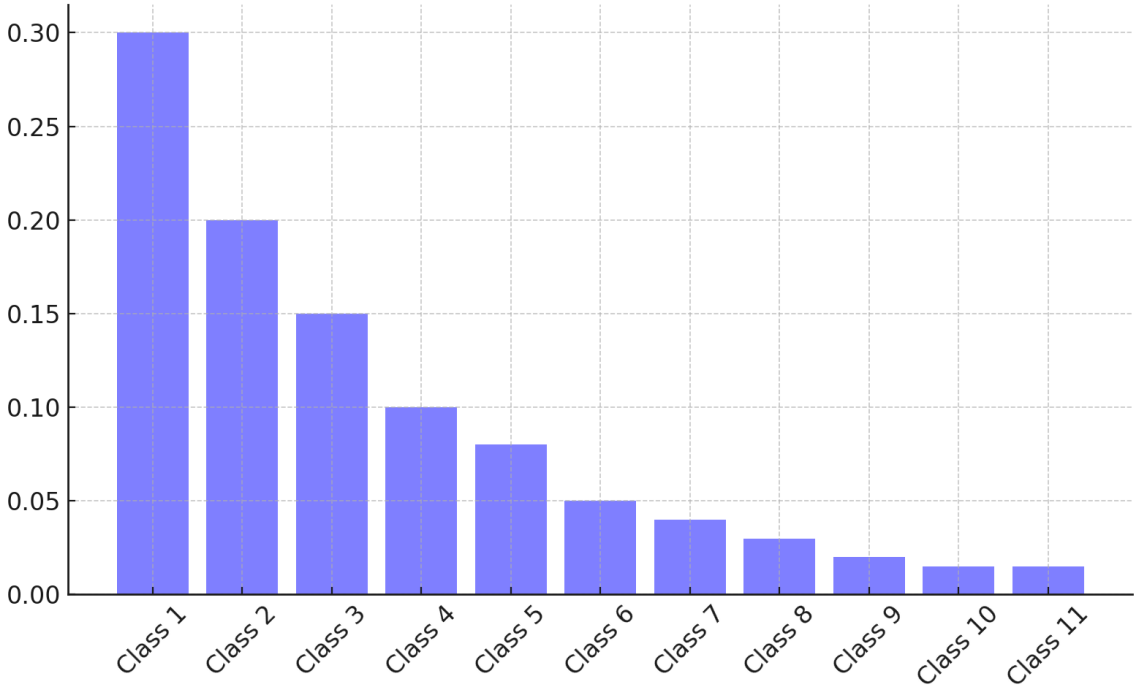
Given the original output of the teacher with sharp probabilities, the student (in theory) can mostly likely be able to learn only which class is correct[1].

[1] https://arxiv.org/pdf/1503.02531

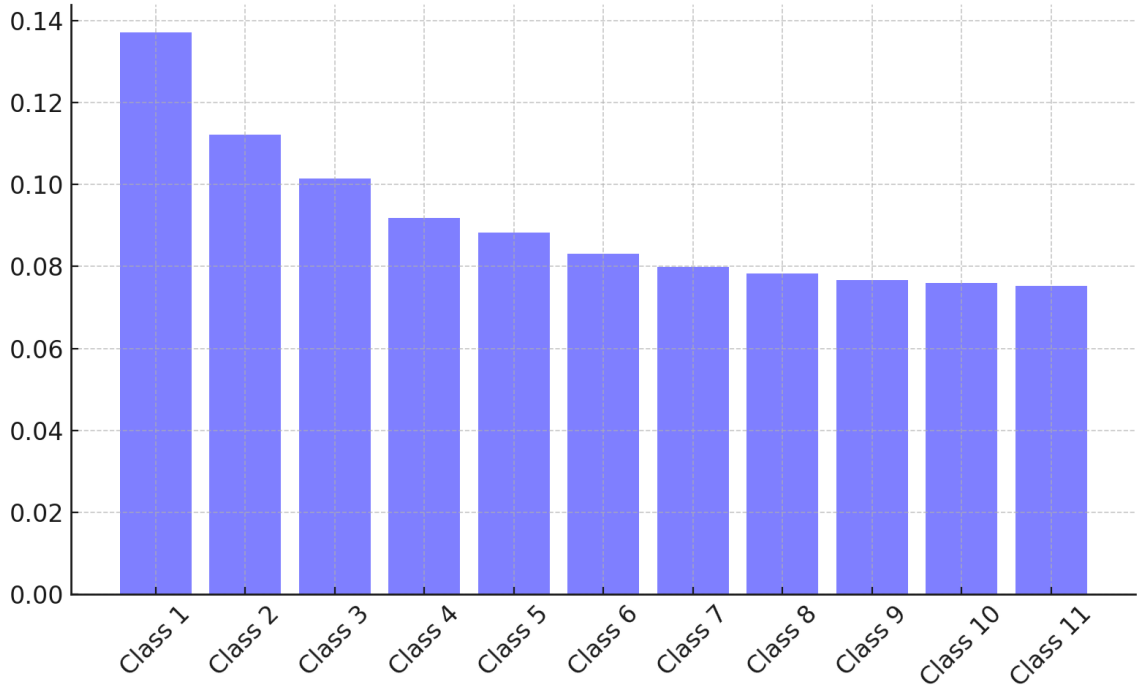
AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ How to distill the knowledge: Generalization



Teacher Original Output.



Teacher Scaled Output.

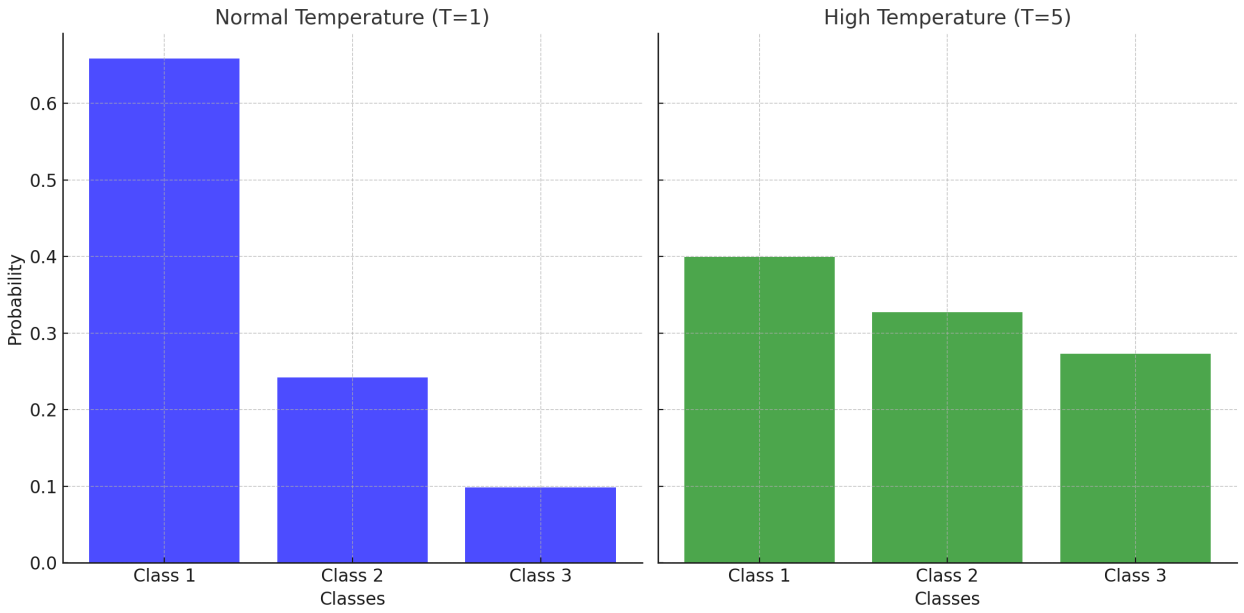
Idea[1]: If somehow we can scale (soften) the teacher output, the relationship between classes is revealed.

[1] https://arxiv.org/pdf/1503.02531

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ How to distill the knowledge: Temperature Scaling



Temperature Scaling: A technique used in machine learning to adjust the sharpness of a model's output probabilities by dividing the logits by a temperature parameter $T > 1$ before applying the Softmax function.

This process softens the probability distribution, making it less confident and highlighting relationships between different outcomes, which can be useful for tasks requiring probabilistic interpretation or better generalization.

Response-based Knowledge

❖ Softmax with Temperature

$$q_i = \frac{\exp(z_i/T)}{\Sigma_j \exp(z_j/T)}$$, where

- q_i : probability using softmax.
- z_i : logit (output before applying softmax).
- T: temperature. Higher T softer probability distribution over classes.

- T = 1: Standard Softmax.
- T>1: Softmax reduces the gap between the top-1 class probability and others, decreasing top-1 and increasing the rest.

E.g. Given logits (raw scores) predicted by Teacher Model for an image of a Cat across 3 classes: $z_{Dog} = 2.0$, $z_{Cat} = 3.0$, $z_{Horse} = 0.5$.

With T = 1:



$q_{Dog} = 0.25$

$q_{Cat} = 0.68$ Top-1

$q_{Horse} = 0.05$

With T = 2 (T>1):



$q_{Dog} = 0.32$ ↑ increase

$q_{Cat} = 0.52$ ↓ decrease

$q_{Horse} = 0.15$ ↑ increase

Response-based Knowledge

❖ Softmax with Temperature

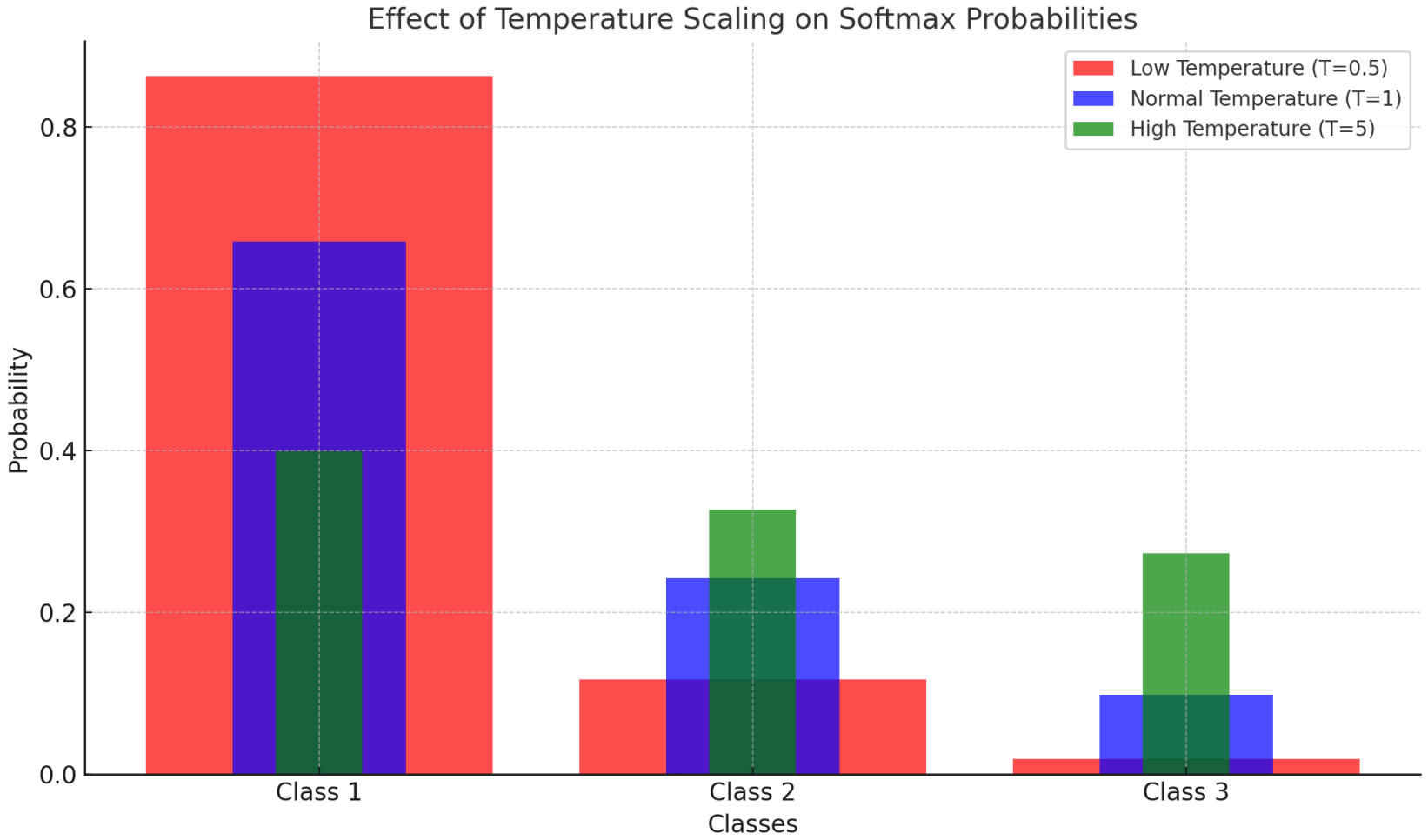


Figure: Illustration of Temperature Scaling technique in different temperature value.

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Distillation loss: Apply temperature scaling

- T = 1: Standard Softmax.
- T>1: Softmax reduces the gap between the top-1 class probability and others, decreasing top-1 and increasing the rest.

$$q_i = \frac{\exp(z_i/T)}{\Sigma_j \exp(z_j/T)}$$, where

- q_i : probability using softmax.
- z_i : logit (output before applying softmax).
- T: Temperature. Higher T softer probability distribution over classes.

$$P_{teacher,i} = \frac{\exp(z_{teacher,i}/T)}{\Sigma_j \exp(z_{teacher,j}/T)}$$

$$P_{student,i} = \frac{\exp(z_{student,i}/T)}{\Sigma_j \exp(z_{student,j}/T)}$$

N = Number of samples in a batch (batch size).

$$L_{KL} = \frac{1}{N}\sum_{n=1}^{N}\sum_{i} P_{teacher,i}(x_n) \cdot (\log P_{teacher,i}(x_n) - \log P_{student,i}(x_n))$$

Response-based Knowledge

❖ Response-based Knowledge Distillation Loss

- Soft Targets Loss:

$$L_{KL} = \frac{1}{N}\sum_{n=1}^{N}\sum_{i} P_{teacher,i}(x_n) \cdot (\log P_{teacher,i}(x_n) - \log P_{student,i}(x_n))$$

- Hard Targets Loss:

$$L_{CE} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{i} y_{n,i} \cdot \log P_{student,i}(x_n)$$

- Combined Loss (Final Loss):

$$L_{KD} = (1-\alpha) \cdot L_{CE}(y, z_s) + \alpha \cdot T^2 \cdot L_{KL}\big(p(z_t, T)||p(z_s, T)\big)$$

Response-based Knowledge

❖ Distillation Loss

- L_{CE} : Cross-entropy loss between true labels y and student logits z_s.
- L_{KL}: Kullback-Leibler divergence between teacher logits $p(z_t, T)$ and student logits $p(z_s, T)$.
- T: Temperature to soften the teacher's logits.
- α: Weight balancing between the two loss components.

where

$$L_{KD} = (1-\alpha) \cdot L_{CE}(y, z_s) + \alpha \cdot T^2 \cdot L_{KL}\big(p(z_t, T) || p(z_s, T)\big)$$

extended formula

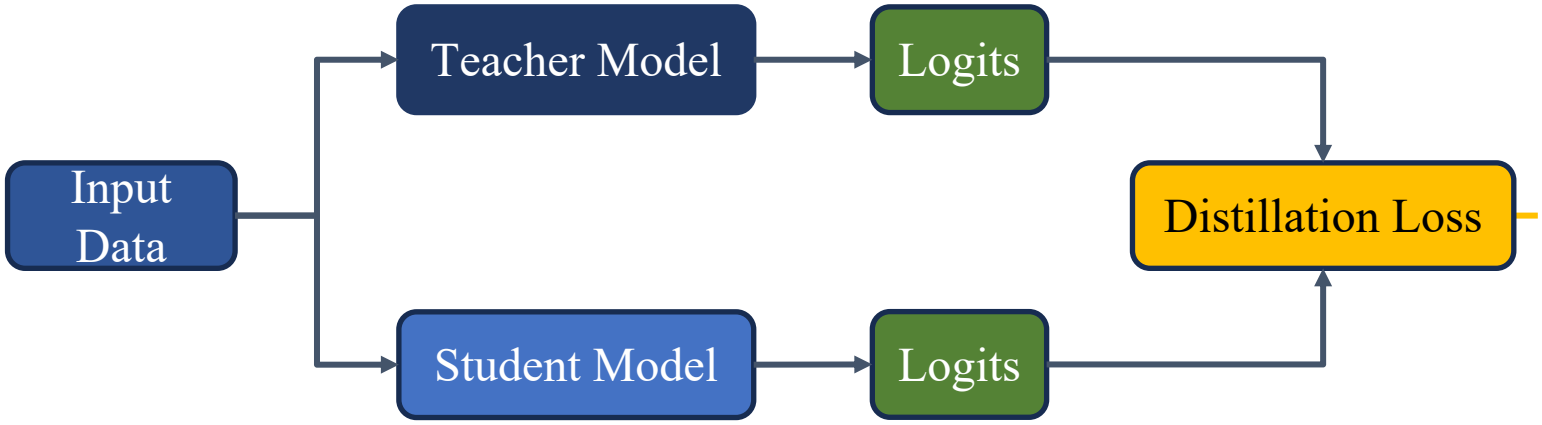


Figure: The generic response-based knowledge distillation. Source: link.

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Introduction

Description: Given an image dataset about weather, build a weather classification model using ResNet and then enhance with the Knowledge Distillation technique.



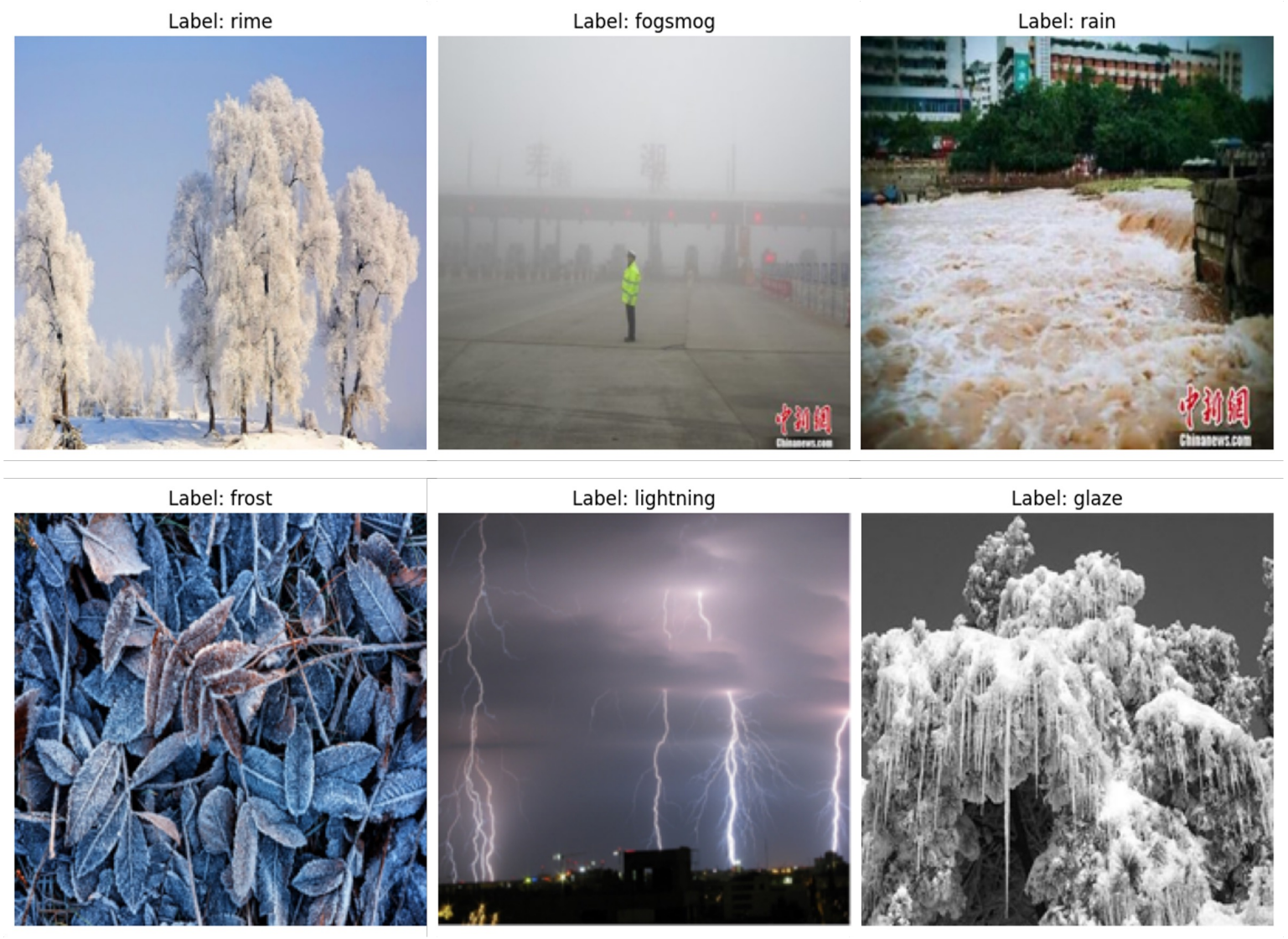
AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Weather Dataset

11 Classes & 6862 images

hail: 591
glaze: 639
rime: 1160
lightning: 377
rainbow: 232
fogsmog: 851
rain: 526
sandstorm: 692
dew: 698
frost: 475
snow: 621



Response-based Knowledge

❖ Weather Dataset

- **Train**: 4,802 samples.
- **Val:** 1,373 samples.
- **Test**: 687 samples.
- **Size**: 224 x 224.
- **Image type**: RGB.





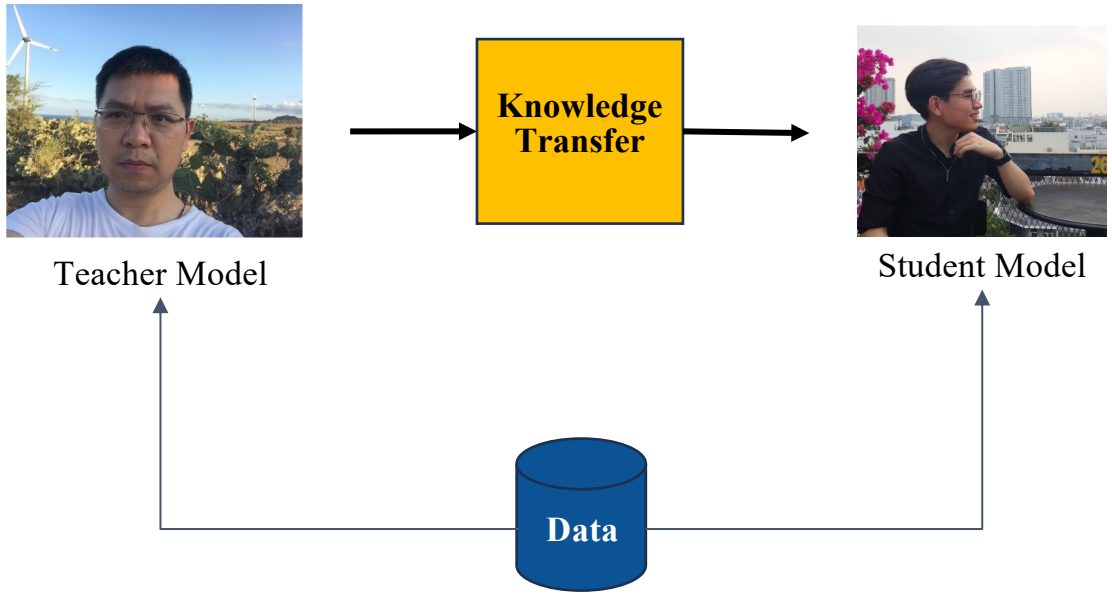




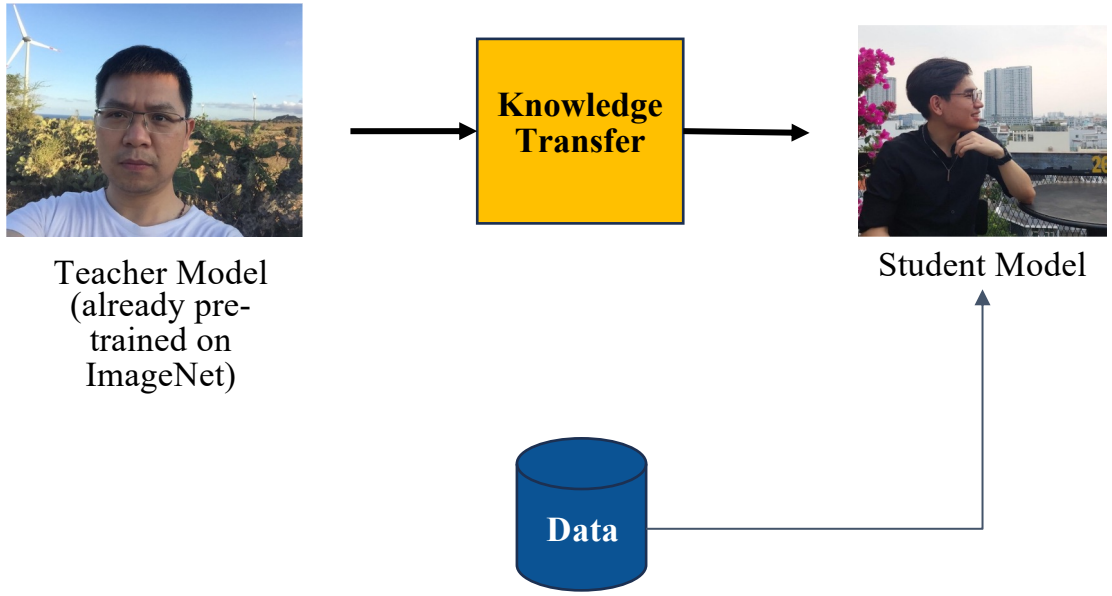


Response-based Knowledge

❖ Knowledge Distillation Scenarios



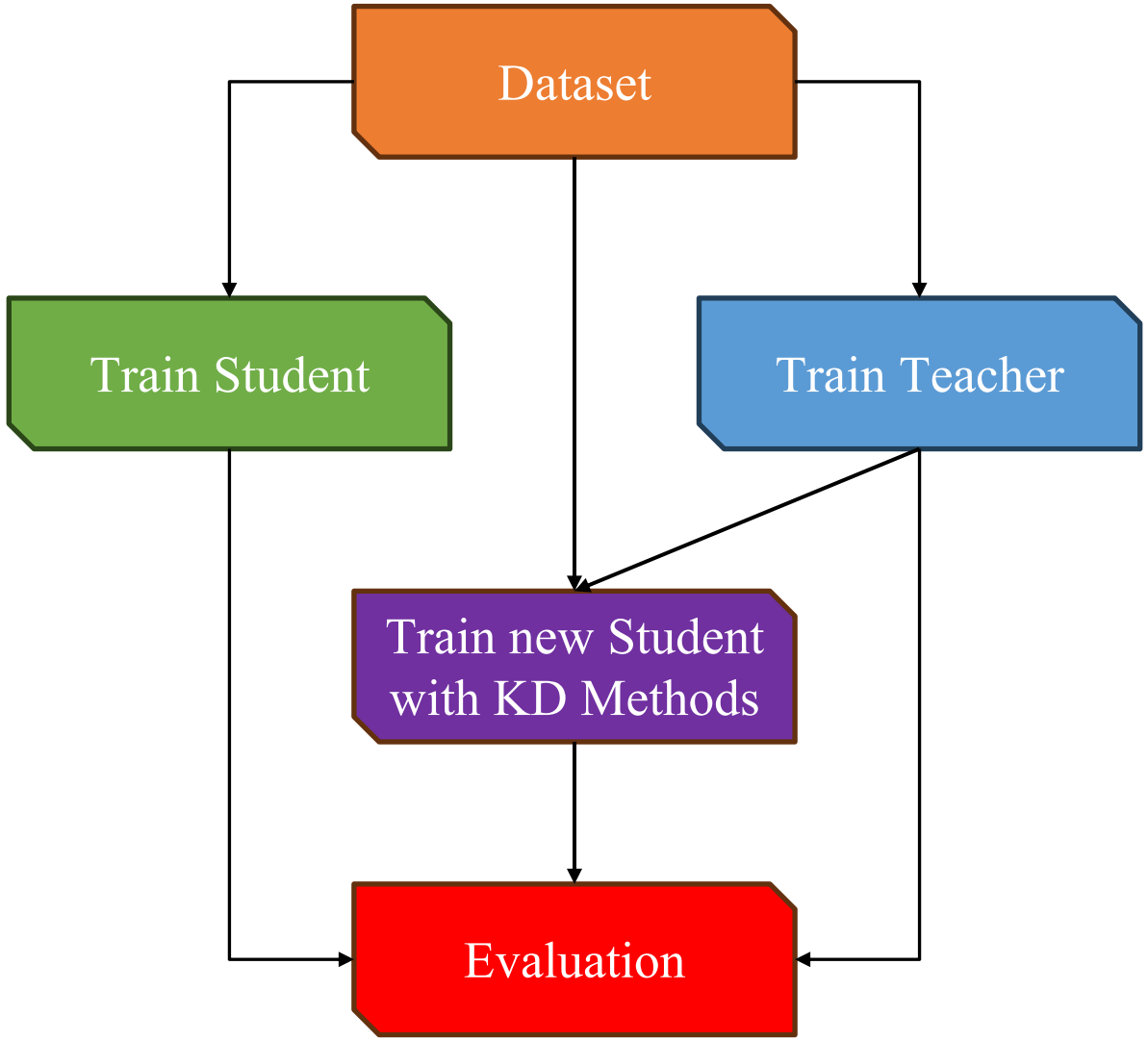
Case 1: Teacher and Student are trained on the same data.



Case 2: Teacher and Student are trained on different data.

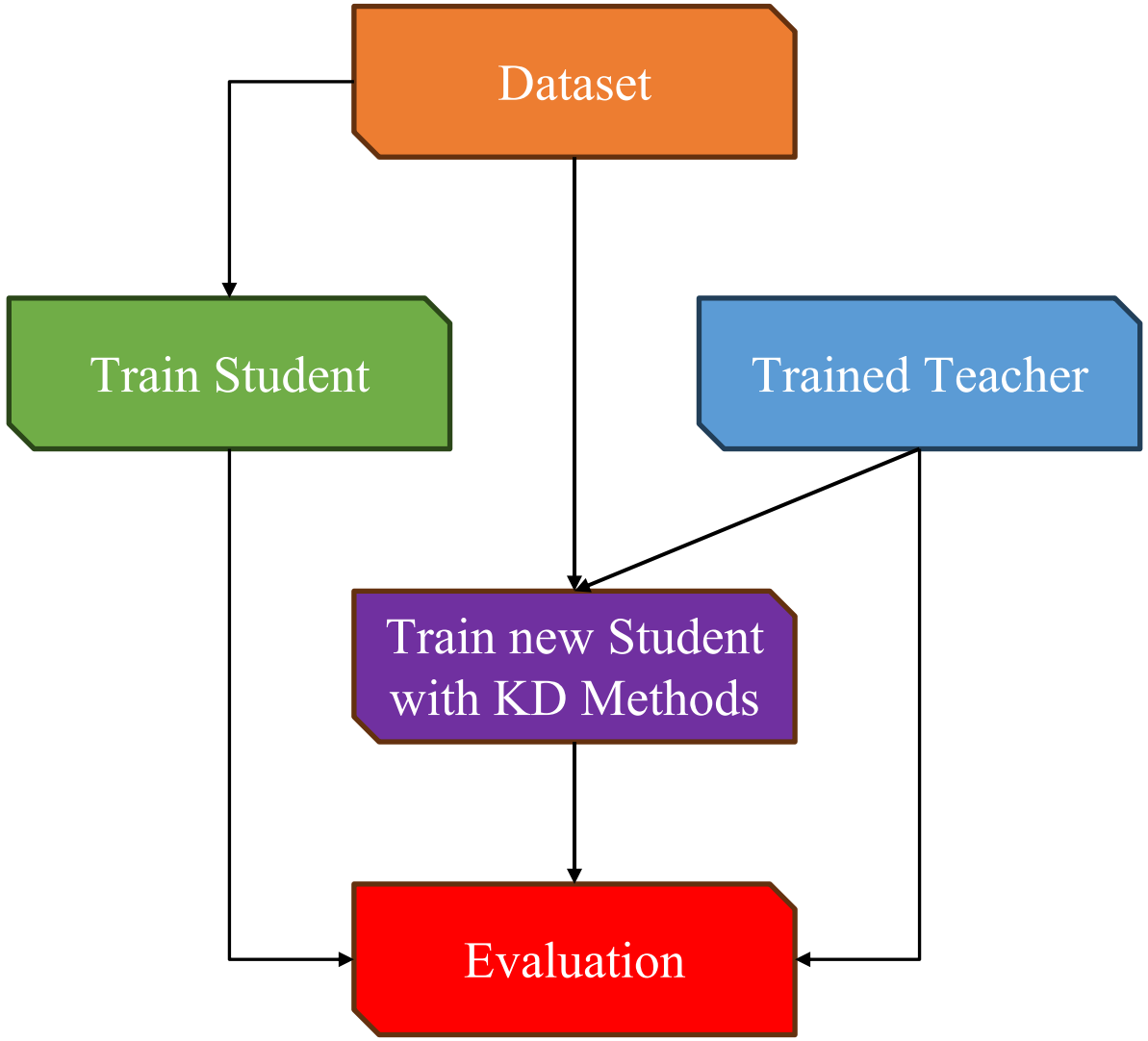
Response-based Knowledge

❖ Pipeline Case 1



Response-based Knowledge

❖ Pipeline Case 2



Response-based Knowledge

❖ ResNet Models

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Response-based Knowledge

❖ Student

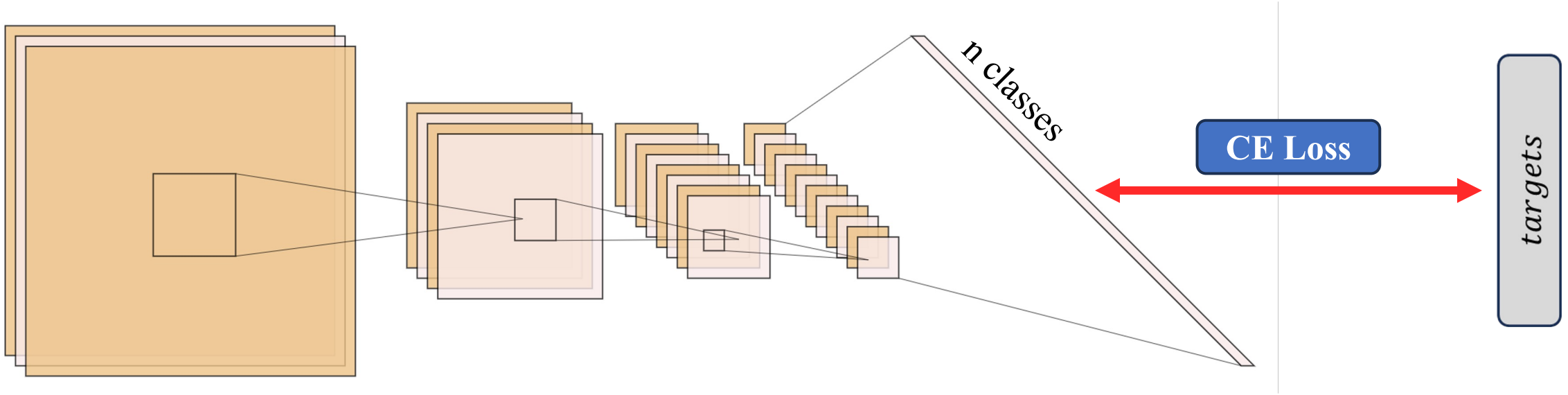


Figure: Example with ResNet18.

Response-based Knowledge

❖ Teacher

Example for ResNet

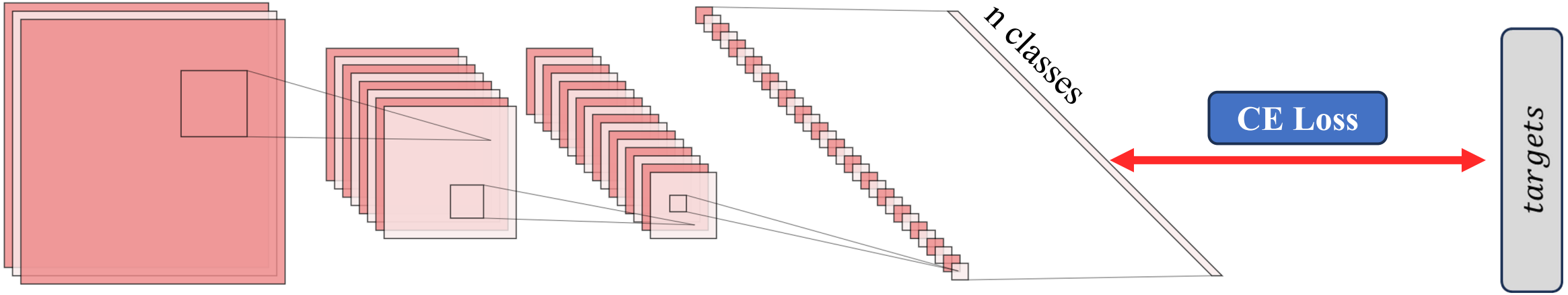
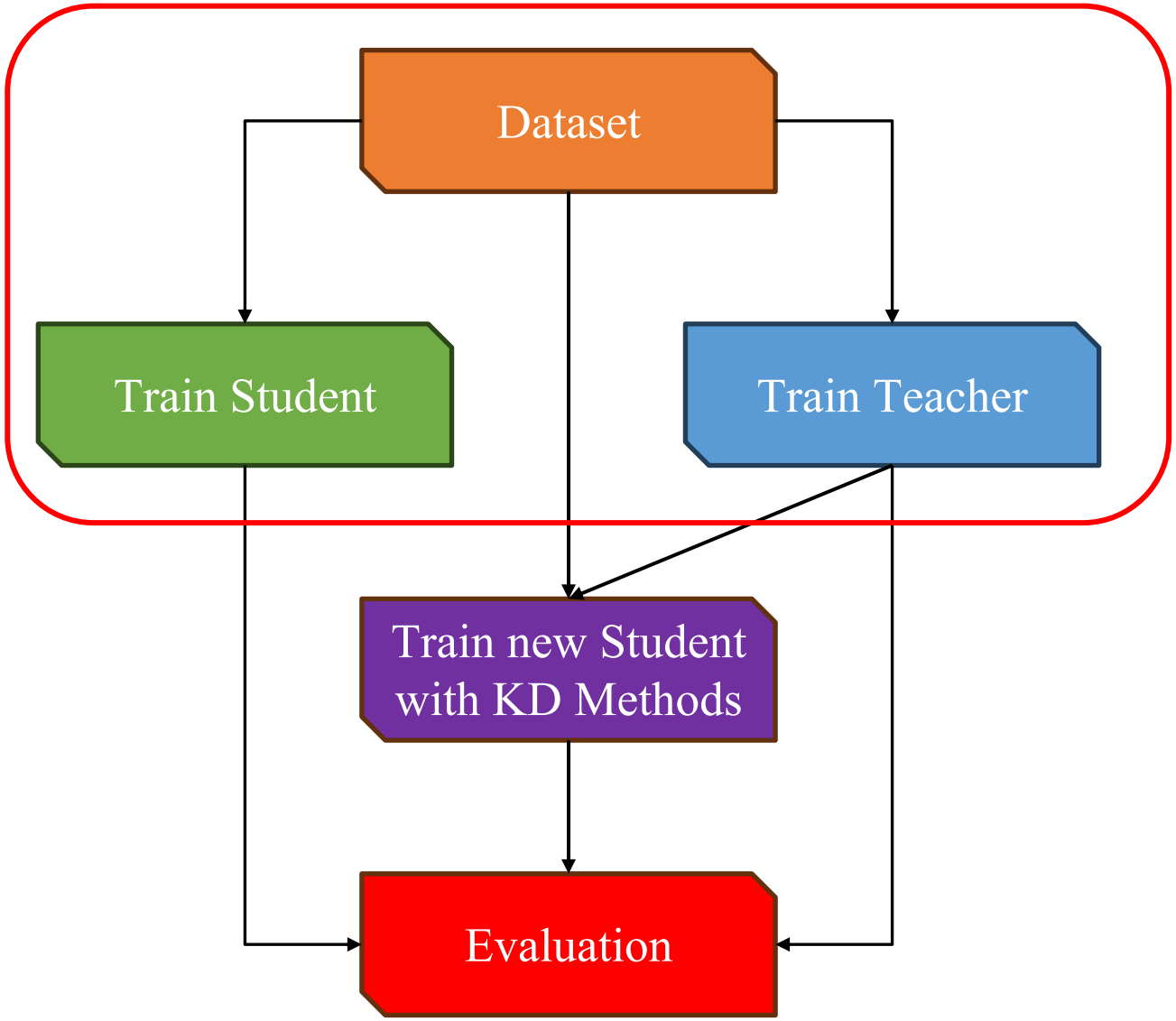


Figure: Example with ResNet34.

Response-based Knowledge

❖ Pipeline Case 1



Response-based Knowledge

❖ Coding Step 1: Import libraries and set random seed

```python
import torch
import torch.nn as nn
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from PIL import Image
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
```

```python
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

seed = 59
set_seed(seed)
```



AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Step 2: Read dataset




```
1 root_dir = 'dataset/weather-dataset/dataset'
2 classes = {
3     label_idx: class_name \
4         for label_idx, class_name in enumerate(
5             sorted(os.listdir(root_dir))
6         )
7 }
```

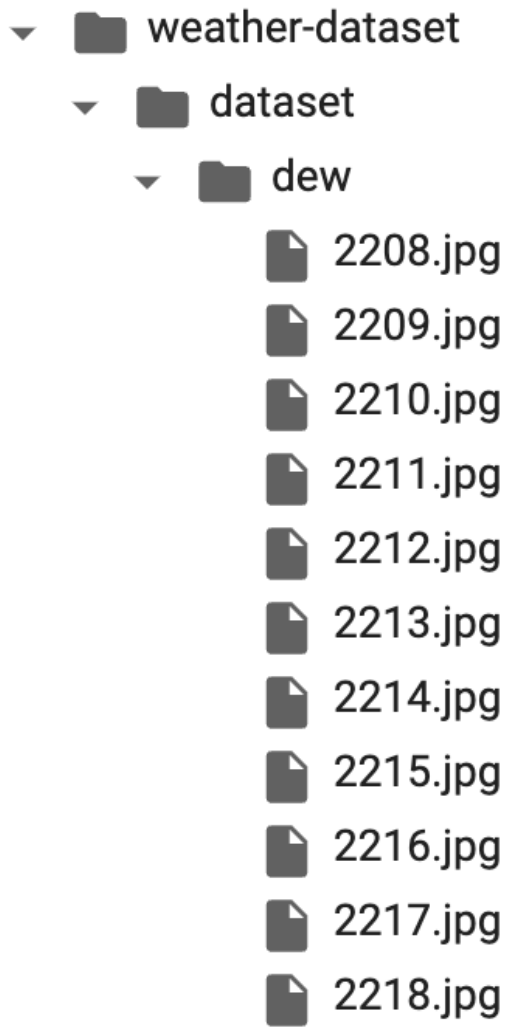
```
1 classes
```

```
{0: 'dew',
 1: 'fogsmog',
 2: 'frost',
 3: 'glaze',
 4: 'hail',
 5: 'lightning',
 6: 'rain',
 7: 'rainbow',
 8: 'rime',
 9: 'sandstorm',
 10: 'snow'}
```

Each folder name is also a class name, thus we could get list of classnames by reading over them.

Response-based Knowledge

❖ Coding Step 2: Read dataset




```
 9 img_paths = []
10 labels = []
11 for label_idx, class_name in classes.items():
12     class_dir = os.path.join(root_dir, class_name)
13     for img_filename in os.listdir(class_dir):
14         img_path = os.path.join(class_dir, img_filename)
15         img_paths.append(img_path)
16         labels.append(label_idx)
```

```
1 img_paths
```

```
['weather-dataset/dataset/dew/2405.jpg',
 'weather-dataset/dataset/dew/2340.jpg',
 'weather-dataset/dataset/dew/2772.jpg',
 'weather-dataset/dataset/dew/2794.jpg',
 'weather-dataset/dataset/dew/2827.jpg',
 'weather-dataset/dataset/dew/2235.jpg',
 'weather-dataset/dataset/dew/2396.jpg',
 'weather-dataset/dataset/dew/2349.jpg',
 'weather-dataset/dataset/dew/2879.jpg',
 'weather-dataset/dataset/dew/2734.jpg',
```

```
1 labels
```

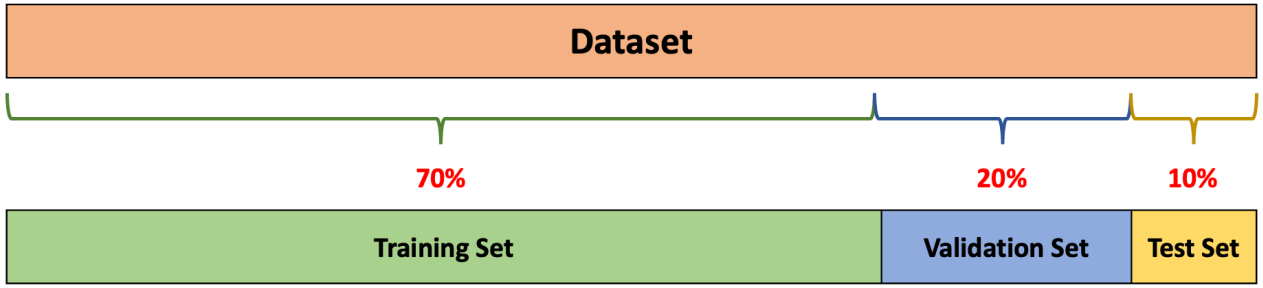
```
[0,
 0,
 0,
 0,
 0,
 0,
 0,
 0,
 0,
 0,
```

We get all image paths as well as their corresponding label.

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Step 3: Train, val, test split




```python
val_size = 0.2
test_size = 0.125
is_shuffle = True

X_train, X_val, y_train, y_val = train_test_split(
    img_paths, labels,
    test_size=val_size,
    random_state=seed,
    shuffle=is_shuffle
)

X_train, X_test, y_train, y_test = train_test_split(
    X_train, y_train,
    test_size=test_size,
    random_state=seed,
    shuffle=is_shuffle
)
```

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Step 4: Create PyTorch Dataset

```python
class WeatherDataset(Dataset):
    def __init__(
        self,
        X, y,
        transform=None
    ):
        self.transform = transform
        self.img_paths = X
        self.labels = y

    def __len__(self):
        return len(self.img_paths)

    def __getitem__(self, idx):
        img_path = self.img_paths[idx]
        img = Image.open(img_path).convert("RGB")

        if self.transform:
            img = self.transform(img)

        return img, self.labels[idx]
```

```python
def transform(img, img_size=(224, 224)):
    img = img.resize(img_size)
    img = np.array(img)[..., :3]
    img = torch.tensor(img).permute(2, 0, 1).float()
    normalized_img = img / 255.0

    return normalized_img
```

```python
train_dataset = WeatherDataset(
    X_train, y_train,
    transform=transform
)
val_dataset = WeatherDataset(
    X_val, y_val,
    transform=transform
)
test_dataset = WeatherDataset(
    X_test, y_test,
    transform=transform
)
```

Declare train, val, test datasets object.

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Step 5: Create DataLoader

```
train_batch_size = 256
test_batch_size = 128

train_loader = DataLoader(
    train_dataset,
    batch_size=train_batch_size,
    shuffle=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=test_batch_size,
    shuffle=False
)
test_loader = DataLoader(
    test_dataset,
    batch_size=test_batch_size,
    shuffle=False
)
```

We must also declare the value of batch size for training and testing.

DataLoader is an iterator, so we can get a sample (image_path, label) and visualize it.

```
train_features, train_labels = next(iter(train_loader))
print(f'Feature batch shape: {train_features.size()}')
print(f'Labels batch shape: {train_labels.size()}')
img = train_features[0].permute(1, 2, 0)
label = train_labels[0].item()
plt.imshow(img)
plt.axis('off')
plt.title(f'Label: {classes[label]}')
plt.show()
```

```
Feature batch shape: torch.Size([256, 3, 224, 224])
Labels batch shape: torch.Size([256])
```



Response-based Knowledge

❖ Coding Step 6: ResNet Architecture

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Response-based Knowledge

❖ Coding Step 6: Implement Residual Block

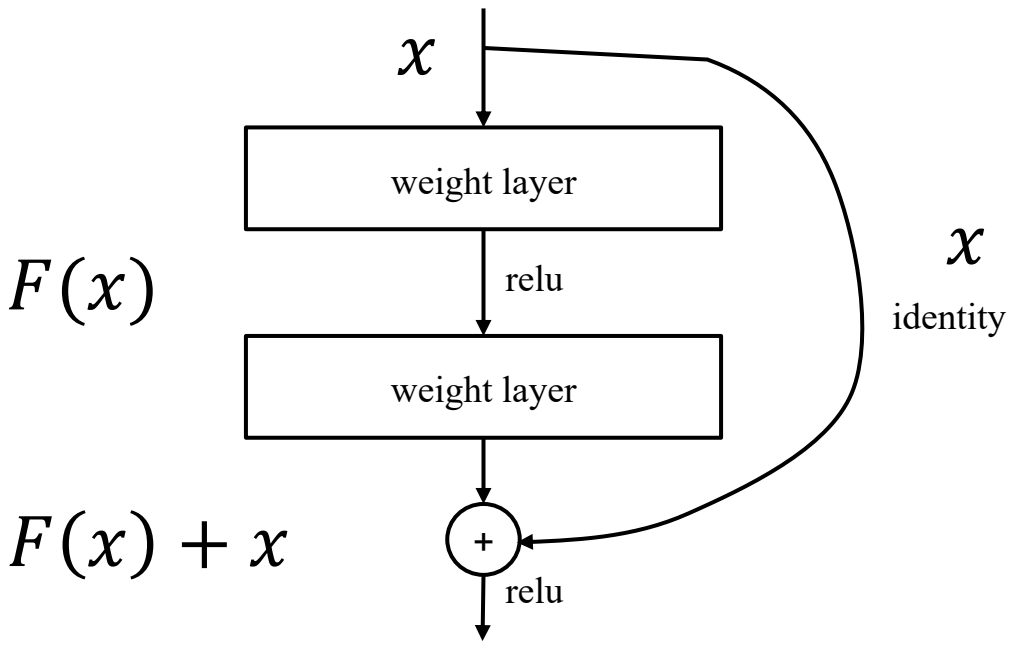


Figure: Residual Block.

```python
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()
        self.conv1 = nn.Conv2d(
            in_channels, out_channels,
            kernel_size=3, stride=stride, padding=1
        )
        self.batch_norm1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(
            out_channels, out_channels,
            kernel_size=3, stride=1, padding=1
        )
        self.batch_norm2 = nn.BatchNorm2d(out_channels)

        self.downsample = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.downsample = nn.Sequential(
                nn.Conv2d(
                    in_channels, out_channels,
                    kernel_size=1, stride=stride
                ),
                nn.BatchNorm2d(out_channels)
            )
        self.relu = nn.ReLU()
```

Response-based Knowledge

❖ Coding Step 6: Implement Residual Block

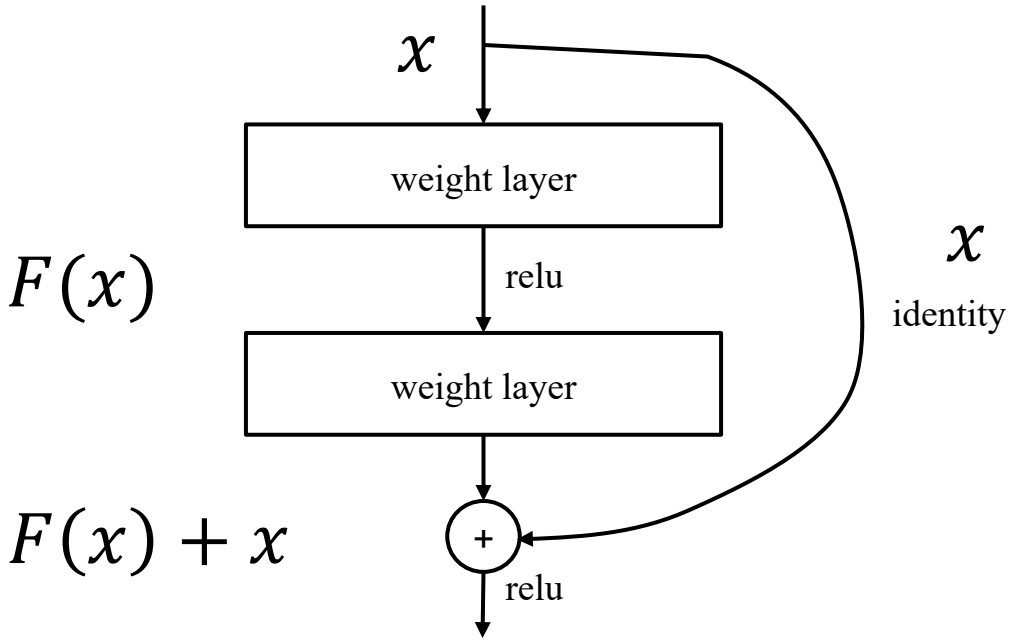
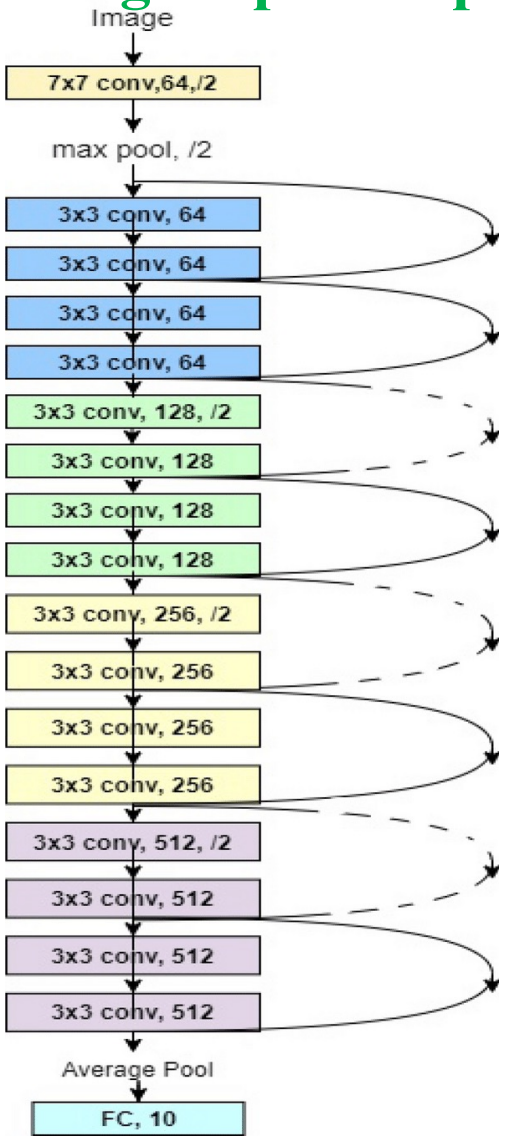


Figure: Residual Block.

```
26      def forward(self, x):
27          shortcut = x.clone()
28          x = self.conv1(x)
29          x = self.batch_norm1(x)
30          x = self.relu(x)
31          x = self.conv2(x)
32          x = self.batch_norm2(x)
33          x += self.downsample(shortcut)
34          x = self.relu(x)
35
36          return x
```

Response-based Knowledge

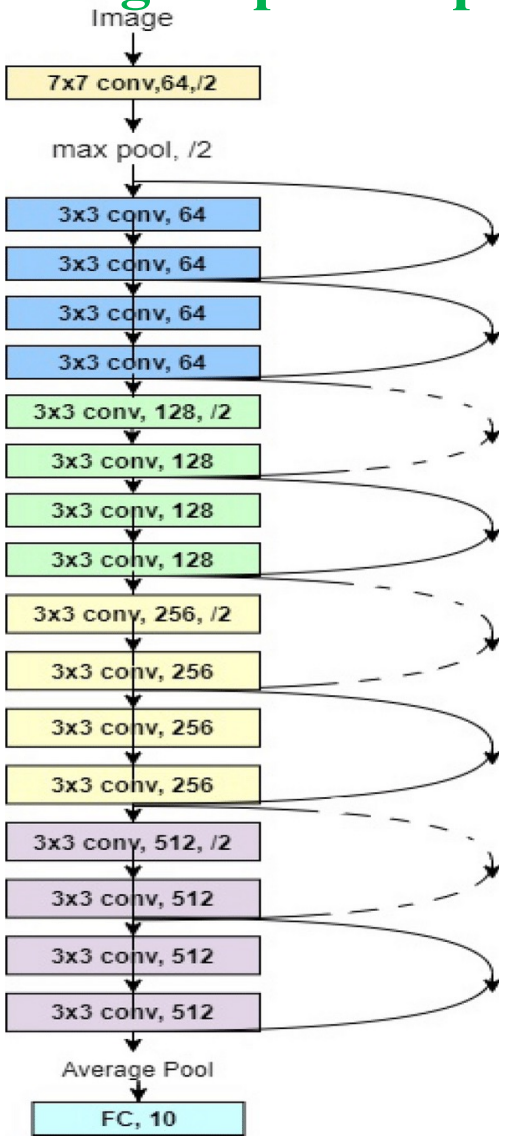
❖ Coding Step 7: Implement ResNet




```python
class ResNet(nn.Module):
    def __init__(self, residual_block, n_blocks_lst, n_classes):
        super(ResNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3)
        self.batch_norm1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.conv2 = self.create_layer(residual_block, 64, 64, n_blocks_lst[0], 1)
        self.conv3 = self.create_layer(residual_block, 64, 128, n_blocks_lst[1], 2)
        self.conv4 = self.create_layer(residual_block, 128, 256, n_blocks_lst[2], 2)
        self.conv5 = self.create_layer(residual_block, 256, 512, n_blocks_lst[3], 2)
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(512, n_classes)
```

Response-based Knowledge

❖ Coding Step 7: Implement ResNet




```python
    def create_layer(self, residual_block, in_channels, out_channels, n_blocks, stride):
        blocks = []
        first_block = residual_block(in_channels, out_channels, stride)
        blocks.append(first_block)

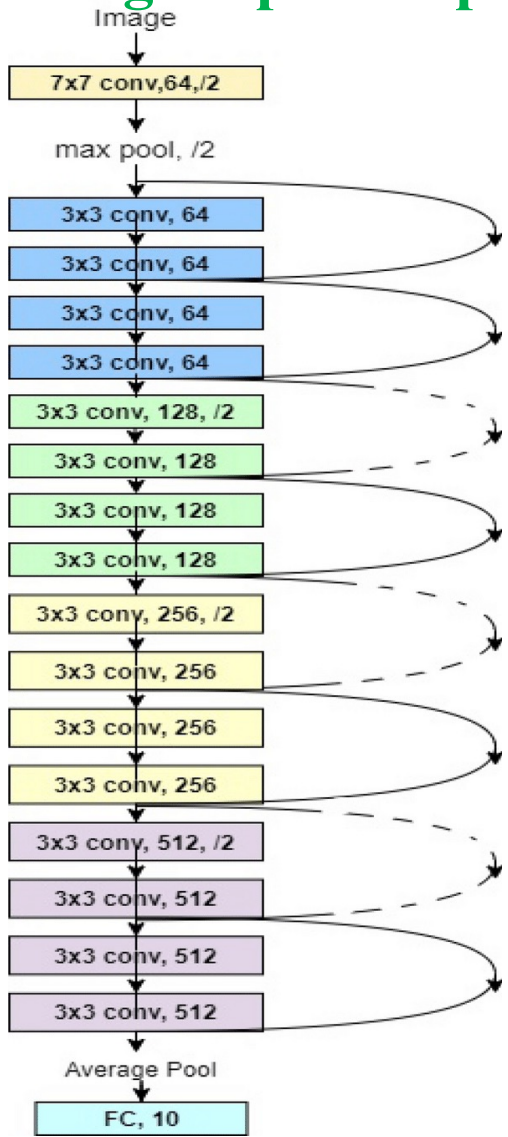
        for idx in range(1, n_blocks):
            block = residual_block(out_channels, out_channels, stride=1)
            blocks.append(block)

        block_sequential = nn.Sequential(*blocks)

        return block_sequential
```

Response-based Knowledge

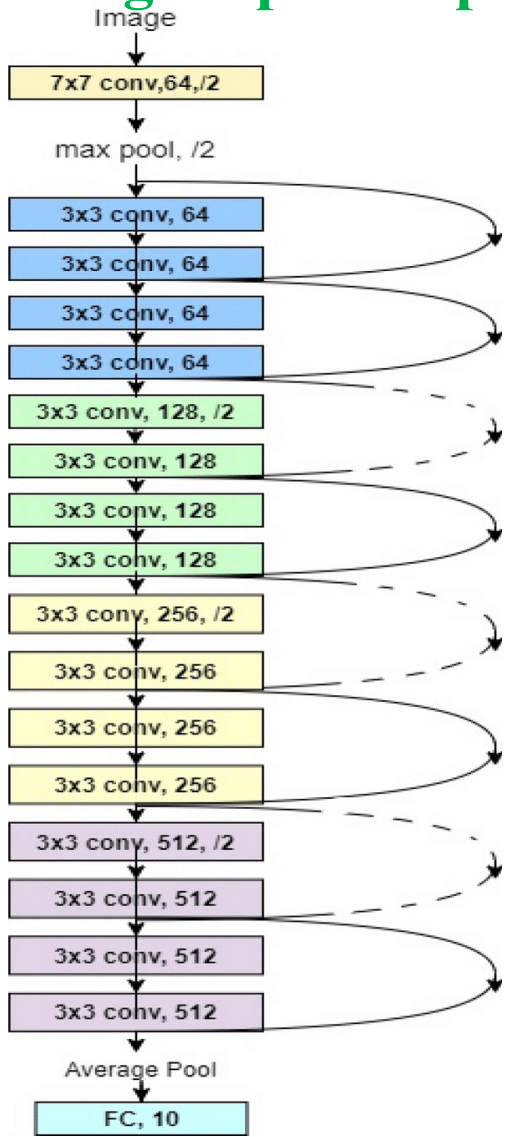
❖ Coding Step 7: Implement ResNet




```
30      def forward(self, x):
31          x = self.conv1(x)
32          x = self.batch_norm1(x)
33          x = self.maxpool(x)
34          x = self.relu(x)
35          x = self.conv2(x)
36          x = self.conv3(x)
37          x = self.conv4(x)
38          x = self.conv5(x)
39          x = self.avgpool(x)
40          x = self.flatten(x)
41          x = self.fc1(x)
42
43          return x
```

Response-based Knowledge

❖ Coding Step 7: Implement ResNet




```python
n_classes = len(list(classes.keys()))
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = ResNet(
    residual_block=ResidualBlock,
    n_blocks_lst=[2, 2, 2, 2],
    n_classes=n_classes
).to(device)
```

```python
model.eval()

dummy_tensor = torch.randn(1, 3, 224, 224).to(device)

with torch.no_grad():
    output = model(dummy_tensor)

print('Output shape:', output.shape)
```

```
Output shape: torch.Size([1, 11])
```

After initialize the model, we should whether it works correctly or not.

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Step 7: ResNet for Teacher and Student

```
n_classes = len(list(classes.keys()))
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = ResNet(
    residual_block=ResidualBlock,
    n_blocks_lst=[2, 2, 2, 2],
    n_classes=n_classes
).to(device)
```

Student Model: ResNet18.

```
n_classes = len(list(classes.keys()))
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = ResNet(
    residual_block=ResidualBlock,
    n_blocks_lst=[3, 4, 6, 3],
    n_classes=n_classes
).to(device)
```

Teacher Model: ResNet18.

Response-based Knowledge

❖ Coding Step 8: Implement training function

```python
def fit(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    scheduler,
    device,
    epochs
):
    train_losses = []
    val_losses = []
    val_accs = []

    for epoch in range(epochs):
        batch_train_losses = []

        model.train()
        for idx, (inputs, labels) in enumerate(train_loader):
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            batch_train_losses.append(loss.item())

        train_loss = sum(batch_train_losses) / len(batch_train_losses)
        train_losses.append(train_loss)

        val_loss, val_acc = evaluate(
            model, val_loader,
            criterion, device
        )
        val_accs.append(val_acc)
        val_losses.append(val_loss)
        scheduler.step()

        print(f'EPOCH {epoch + 1}:\tTrain loss: {train_loss:.4f}\tVal loss: {val_loss:.4f}\tVal acc: {val_acc:.4f}')

    return train_losses, val_losses, val_accs
```

Response-based Knowledge

❖ Coding Step 9: Training

```python
lr = 1e-2
epochs = 30

def lr_lambda(epoch, warmup_epochs=5, total_epochs=30,
              init_scale=0.1, min_scale=0.3):
    scale_range = 1.0 - min_scale

    if epoch < warmup_epochs:
        warmup_factor = epoch / warmup_epochs
        return init_scale + (1.0 - init_scale) * warmup_factor

    decay_factor = (total_epochs - epoch) / (total_epochs - warmup_epochs)
    return min_scale + scale_range * max(0.0, decay_factor)


criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=lr
)

scheduler = LambdaLR(optimizer, lr_lambda=lr_lambda)
```

```python
train_losses, val_losses, val_accs = fit(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    scheduler,
    device,
    epochs
)
```

```
EPOCH 1:        Train loss: 2.2829      Val loss: 2.3994
EPOCH 2:        Train loss: 1.8316      Val loss: 2.3707
EPOCH 3:        Train loss: 1.4704      Val loss: 1.4859
EPOCH 4:        Train loss: 1.2538      Val loss: 1.4205
EPOCH 5:        Train loss: 1.1788      Val loss: 1.3416
EPOCH 6:        Train loss: 1.0918      Val loss: 1.2270
EPOCH 7:        Train loss: 0.9752      Val loss: 1.3701
EPOCH 8:        Train loss: 0.8907      Val loss: 2.2087
EPOCH 9:        Train loss: 0.8120      Val loss: 1.7150
EPOCH 10:       Train loss: 0.7552      Val loss: 1.3199
```

AI VIET NAM
@aivietnam.edu.vn

Response-based Knowledge

❖ Coding Step 10: Evaluation

```python
def evaluate(model, dataloader, criterion, device):
    model.eval()
    correct = 0
    total = 0
    losses = []
    with torch.no_grad():
        for inputs, labels in dataloader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            losses.append(loss.item())
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    loss = sum(losses) / len(losses)
    acc = correct / total

    return loss, acc
```

```python
val_loss, val_acc = evaluate(
    model,
    val_loader,
    criterion,
    device
)
test_loss, test_acc = evaluate(
    model,
    test_loader,
    criterion,
    device
)

print('Evaluation on val/test dataset')
print('Val accuracy: ', val_acc)
print('Test accuracy: ', test_acc)
```

```
Evaluation on val/test dataset
Val accuracy:  0.7654770575382375
Test accuracy:  0.7540029112081513
```

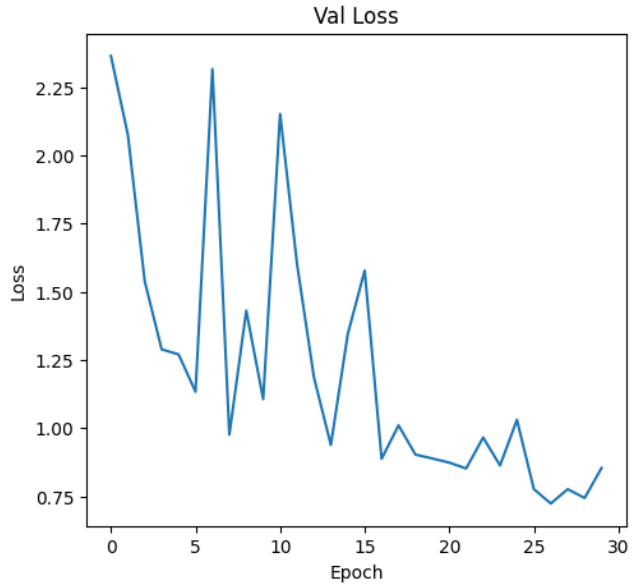
AI VIET NAM
@aivietnam.edu.vn

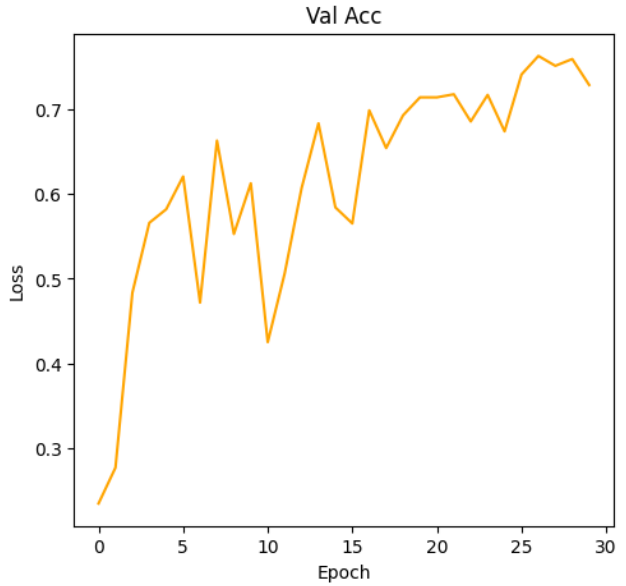
Response-based Knowledge

❖ Results

Student

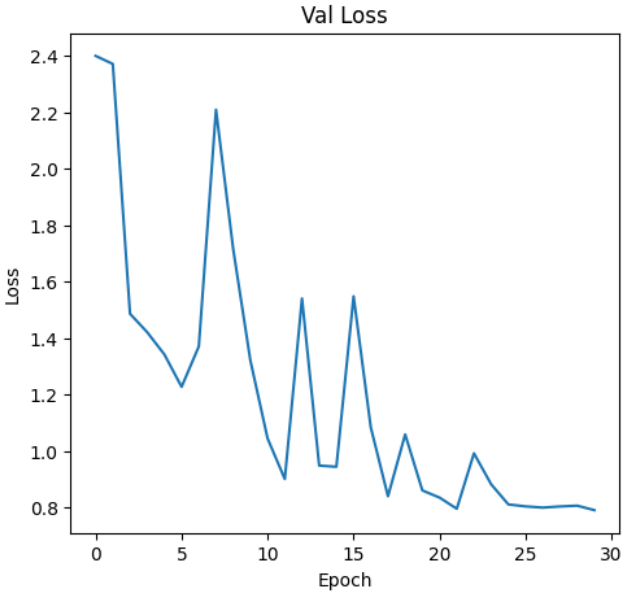
```
Val accuracy:   0.7283
Test accuracy:  0.7278
```

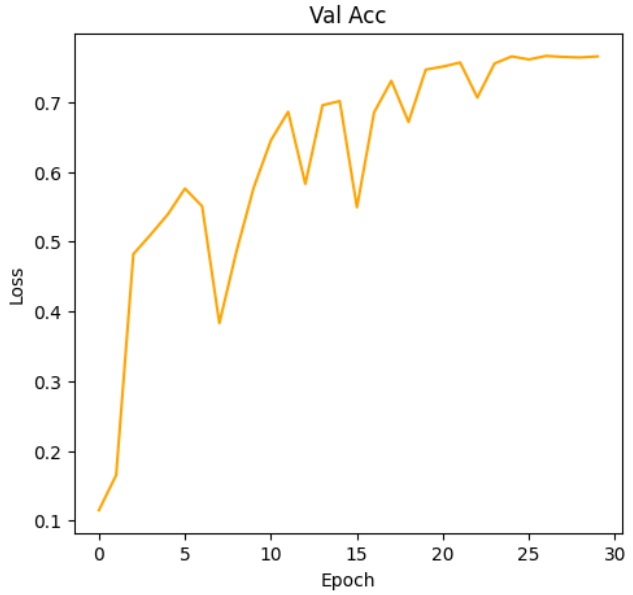




Teacher

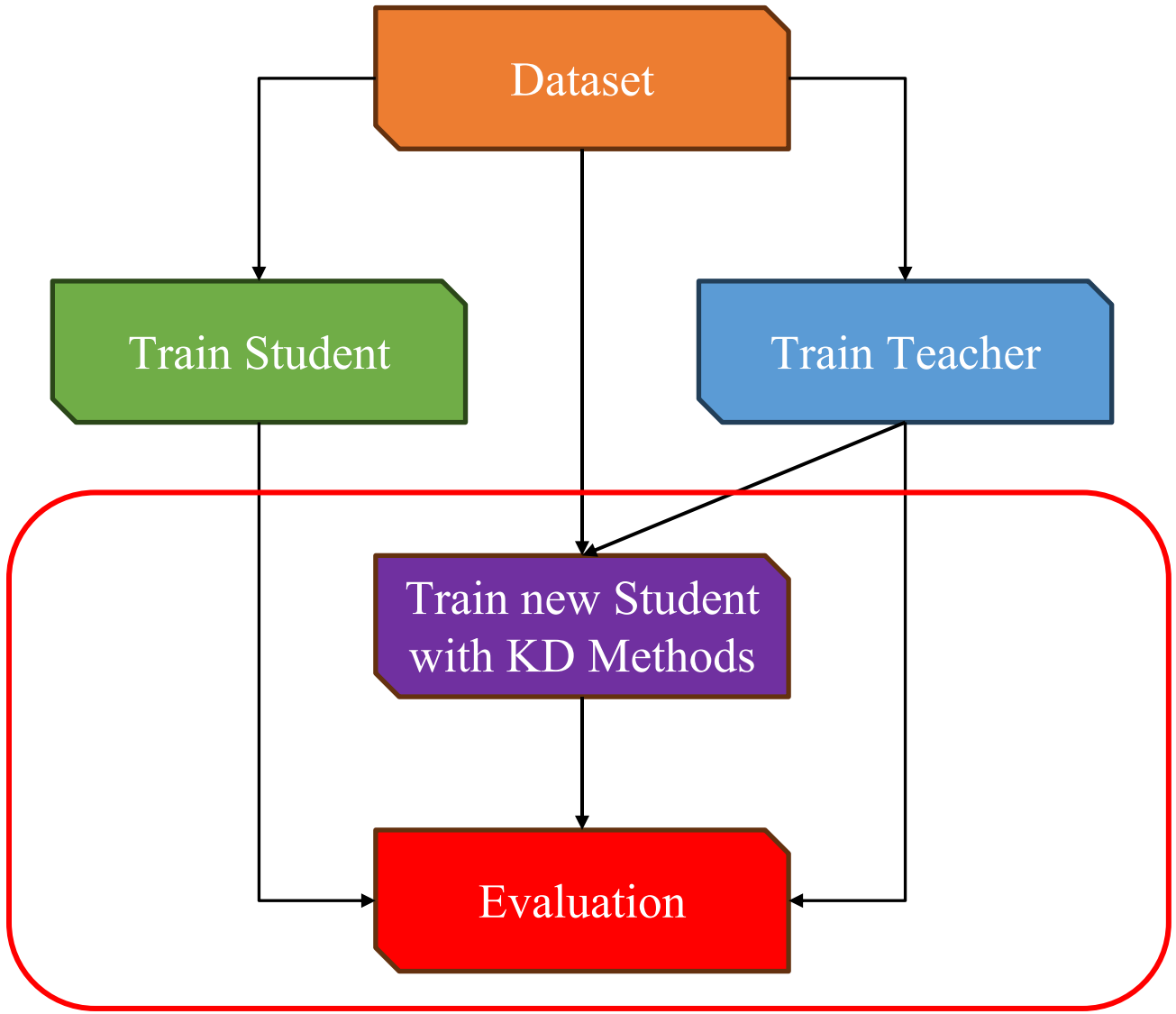
```
Val accuracy:   0.7655
Test accuracy:  0.7540
```





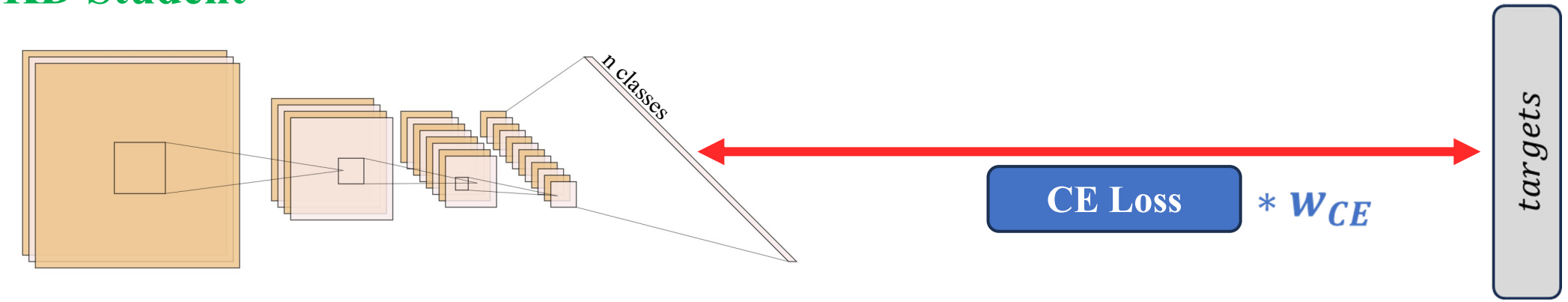
Response-based Knowledge

❖ Pipeline Case 1



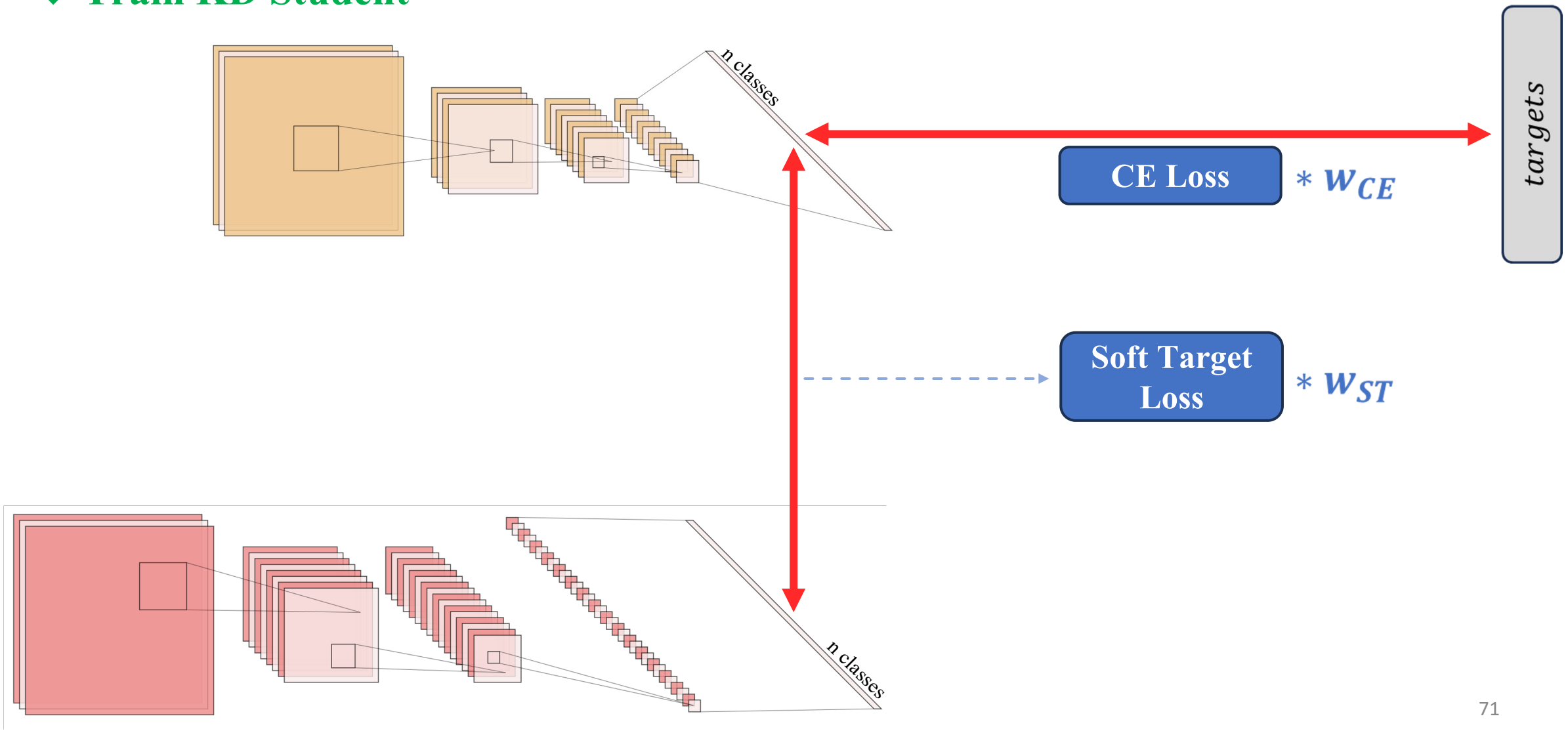
Response-based Knowledge

❖ Train KD Student



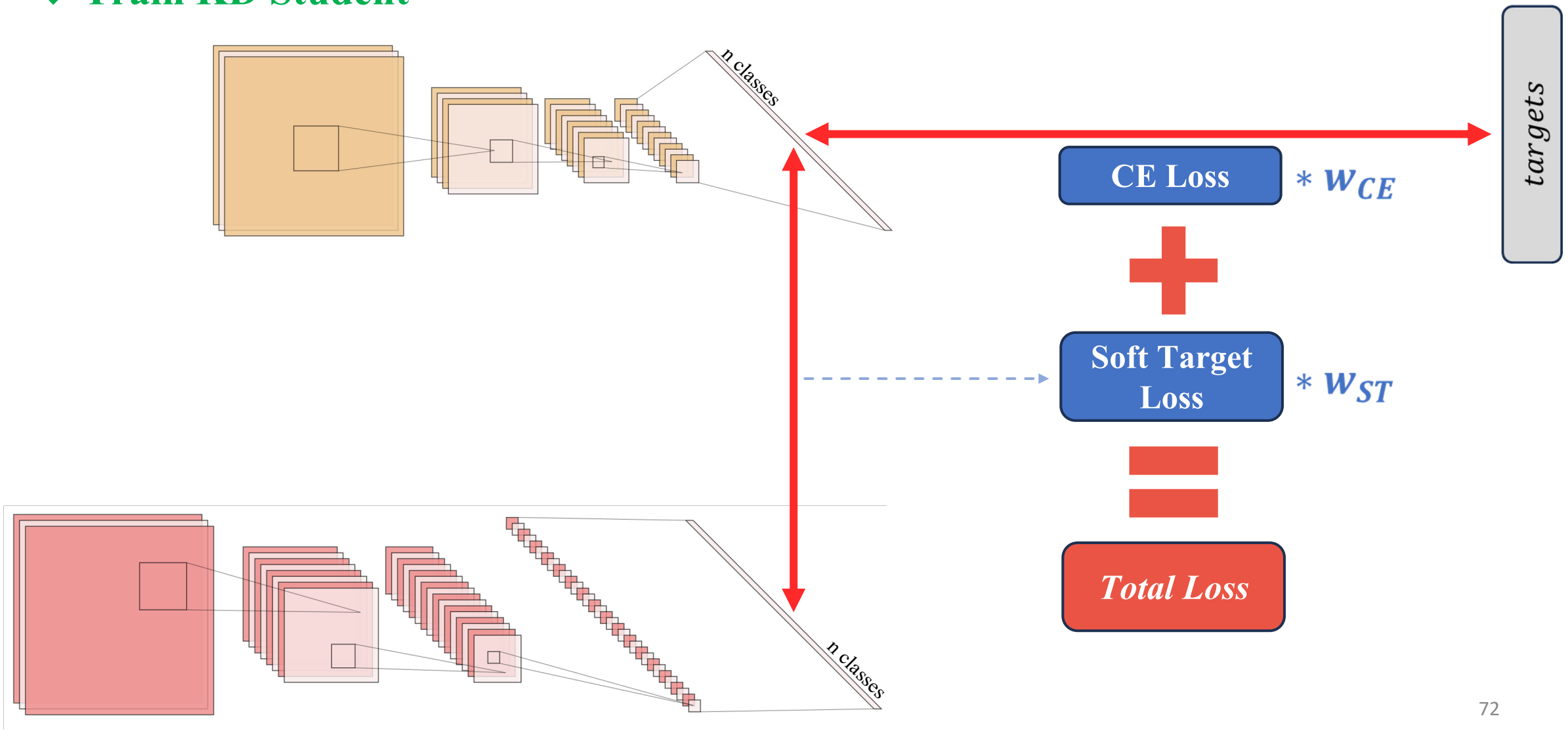
Response-based Knowledge

❖ Train KD Student



Response-based Knowledge

❖ Train KD Student



Response-based Knowledge

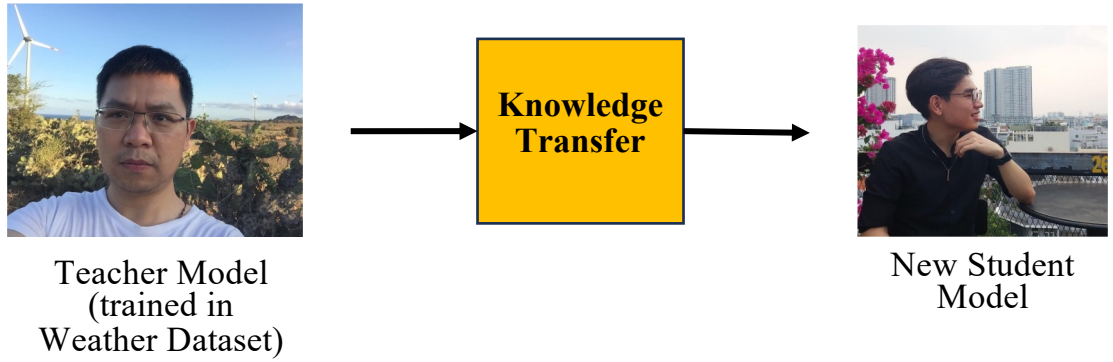
❖ Coding Step 1: Initialize Teacher and New Student

```
n_classes = len(list(classes.keys()))
device = 'cuda' if torch.cuda.is_available() else 'cpu'

teacher_model = ResNet(
    residual_block=ResidualBlock,
    n_blocks_lst=[3, 4, 6, 3],
    n_classes=n_classes
).to(device)

teacher_model.load_state_dict(
    torch.load(
        'teacher_wt.pt',
        map_location=device
    )
)

student_model = ResNet(
    residual_block=ResidualBlock,
    n_blocks_lst=[2, 2, 2, 2],
    n_classes=n_classes
).to(device)
```



Response-based Knowledge

❖ Coding Step 2: Implement training function

```python
def fit(
    teacher,
    student,
    T,
    soft_target_loss_weight,
    ce_loss_weight,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    scheduler,
    device,
    epochs
):
    train_losses = []
    val_losses = []
    val_accs = []
    teacher.eval()
    for epoch in range(epochs):
        batch_train_losses = []

        student.train()
        for idx, (inputs, labels) in enumerate(train_loader):
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()

            with torch.no_grad():
                teacher_logits = teacher(inputs)
            student_logits = student(inputs)

            soft_targets = nn.functional.softmax(teacher_logits / T, dim=-1)
            soft_prob = nn.functional.log_softmax(student_logits / T, dim=-1)

            soft_targets_loss = torch.sum(soft_targets * (soft_targets.log() - soft_prob)) / soft_prob.size()[0] * (T**2)
            label_loss = criterion(student_logits, labels)

            loss = soft_target_loss_weight * soft_targets_loss + ce_loss_weight * label_loss

            loss.backward()
            optimizer.step()

            batch_train_losses.append(loss.item())

        train_loss = sum(batch_train_losses) / len(batch_train_losses)
        train_losses.append(train_loss)

        val_loss, val_acc = evaluate(
            student, val_loader,
            criterion, device
        )
        val_accs.append(val_acc)
        val_losses.append(val_loss)
        scheduler.step()
```

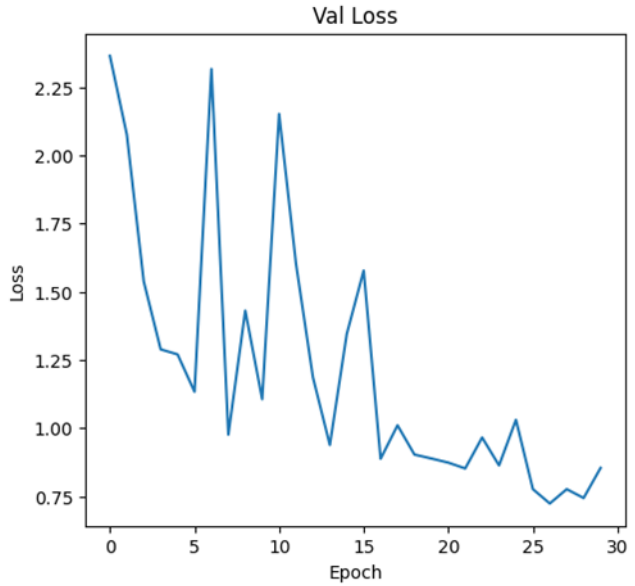
AI VIET NAM
@aivietnam.edu.vn

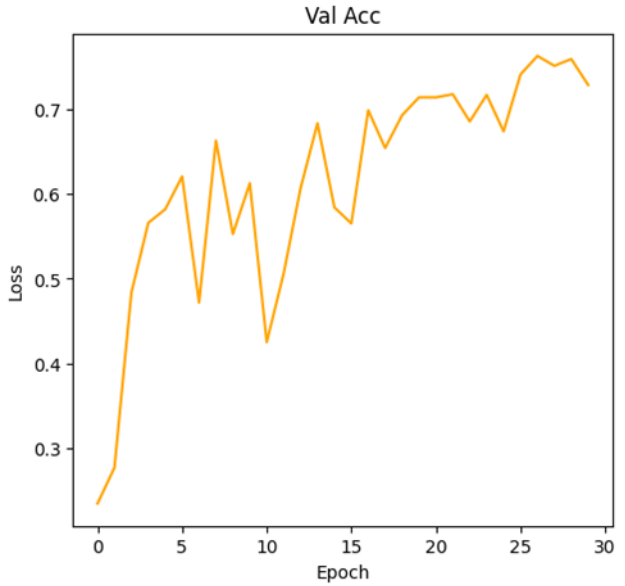
Response-based Knowledge

❖ Results

Student

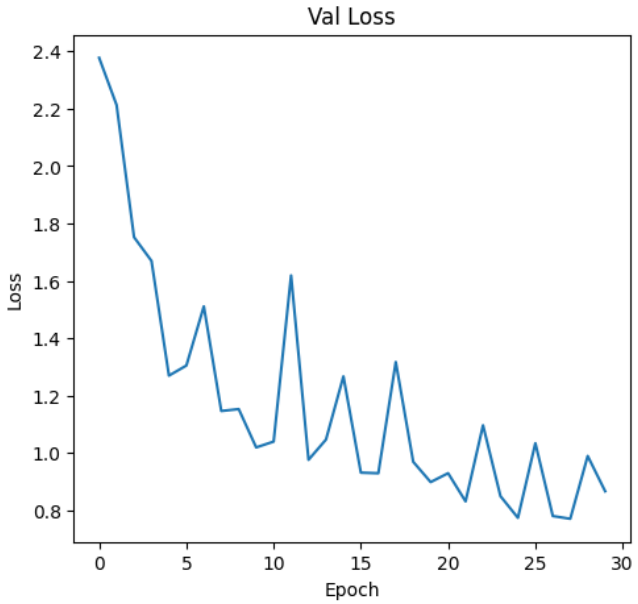
```
Val accuracy:   0.7283
Test accuracy:  0.7278
```

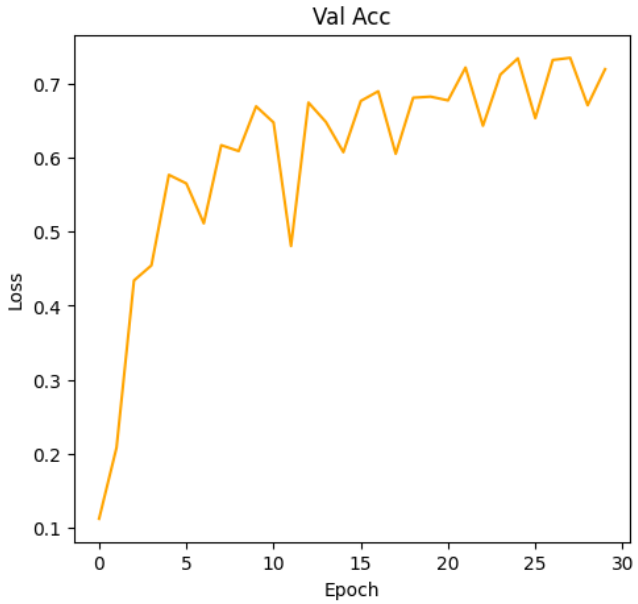




KD with Teacher on the same dataset

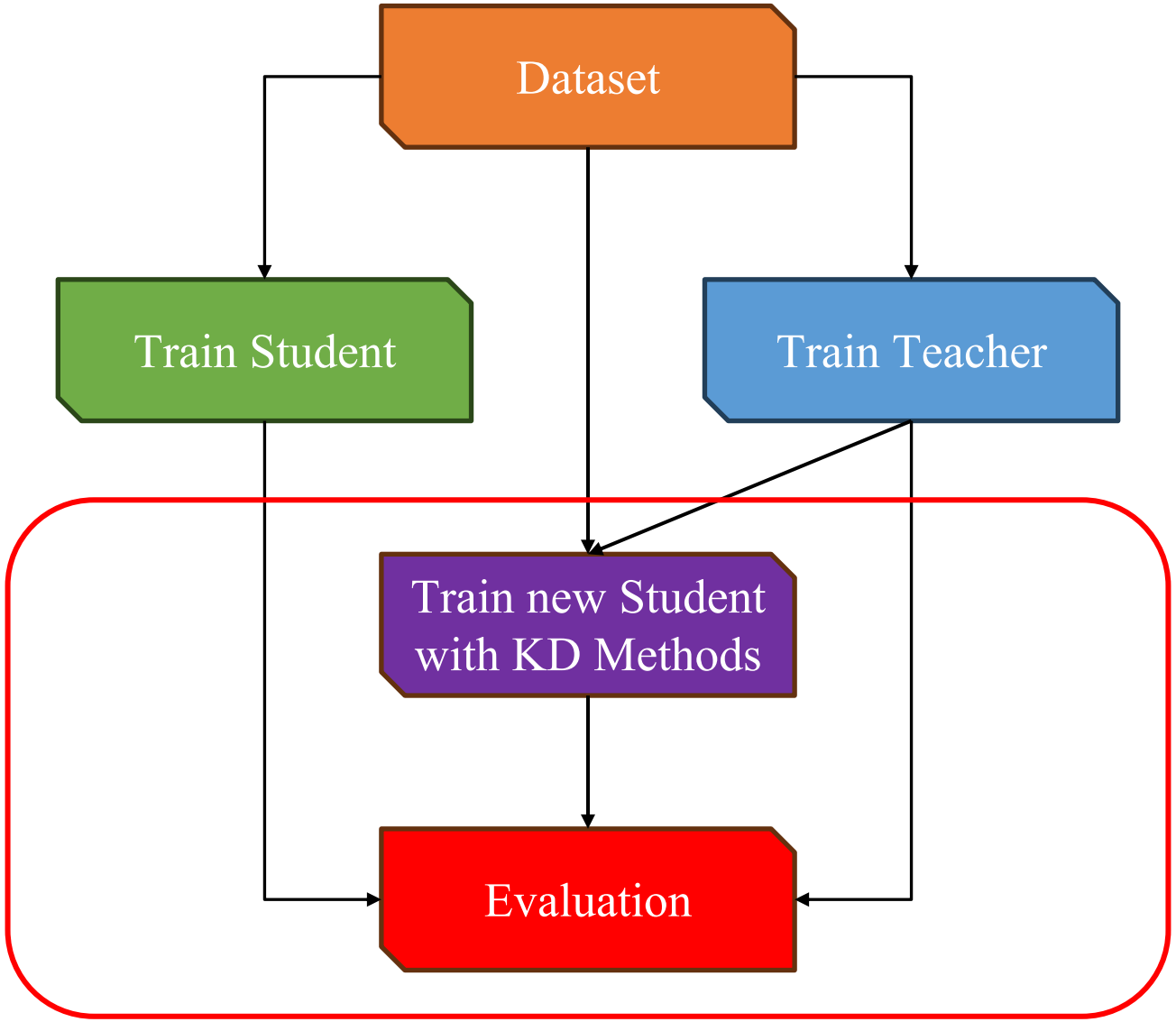
```
Val accuracy:   0.7196
Test accuracy:  0.7351
```





Response-based Knowledge

❖ Pipeline Case 2



AI VIET NAM
@aivietnam.edu.vn

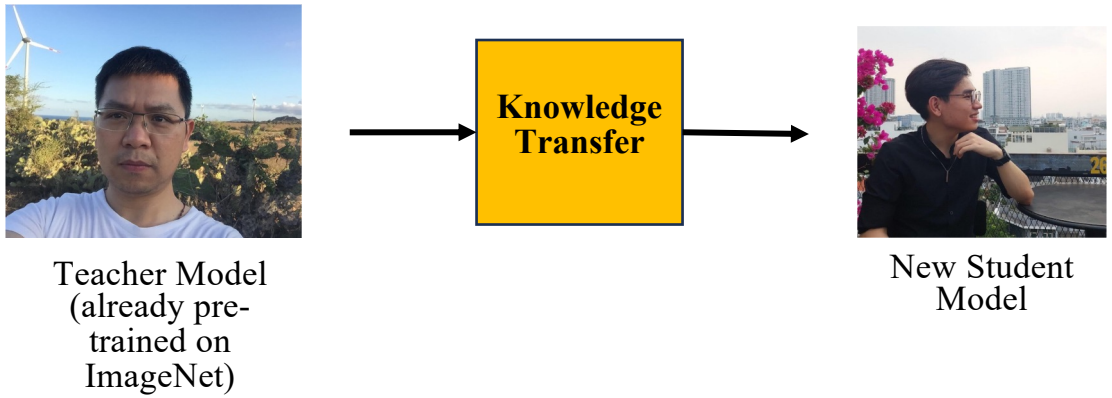
Response-based Knowledge

❖ Initialize Teacher (pre-trained on ImageNet) and New Student

```python
n_classes = len(list(classes.keys()))
device = 'cuda' if torch.cuda.is_available() else 'cpu'

teacher_model = timm.create_model(
    'resnet34',
    pretrained=True,
    num_classes=n_classes
).to(device)

student_model = ResNet(
    residual_block=ResidualBlock,
    n_blocks_lst=[2, 2, 2, 2],
    n_classes=n_classes
).to(device)
```



Teacher Model (already pre-trained on ImageNet)

New Student Model

Loading pre-trained ResNet34 on ImageNet using timm library.

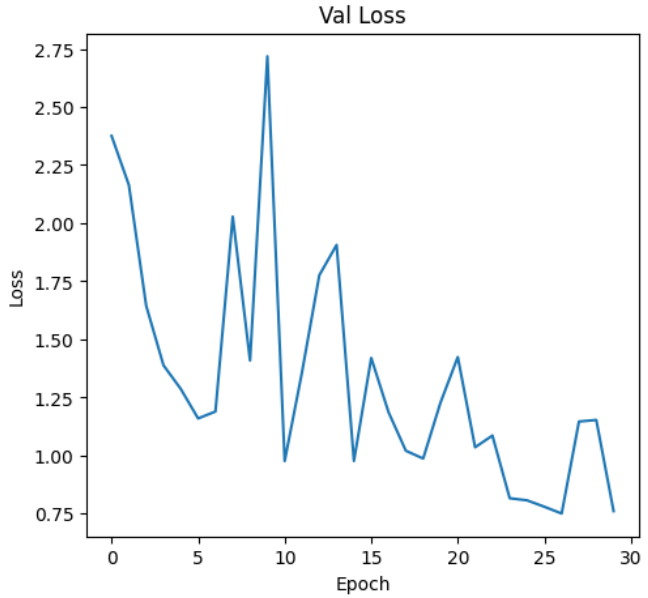
AI VIET NAM
@aivietnam.edu.vn

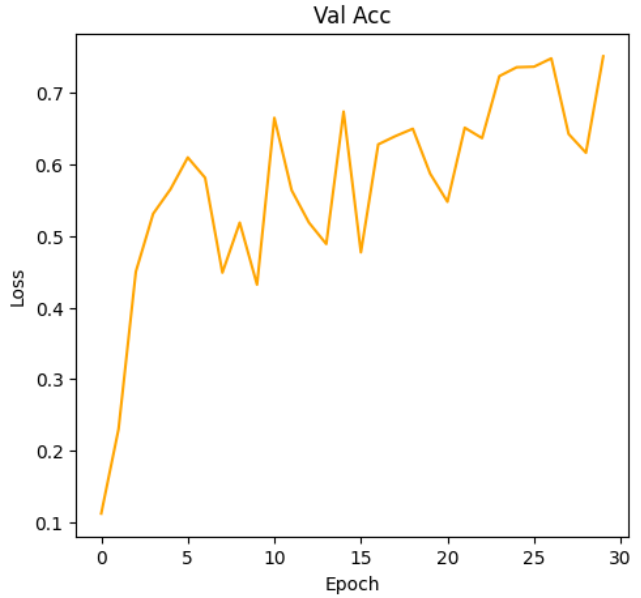
Response-based Knowledge

❖ Results

KD with Teacher pretrained on ImageNet

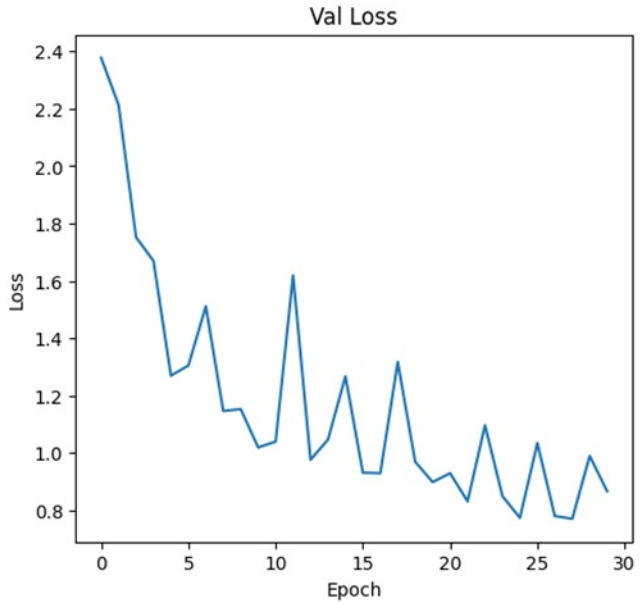
```
Val accuracy:  0.7509
Test accuracy:  0.7467
```

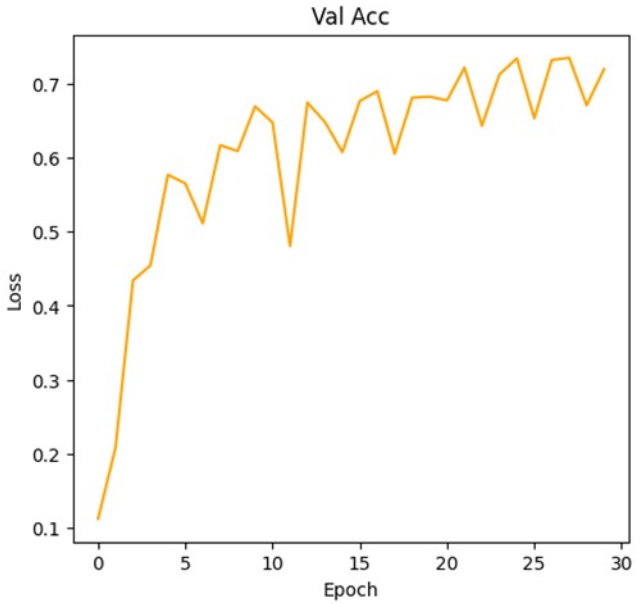




KD with Teacher on the same dataset

```
Val accuracy:  0.7196
Test accuracy:  0.7351
```





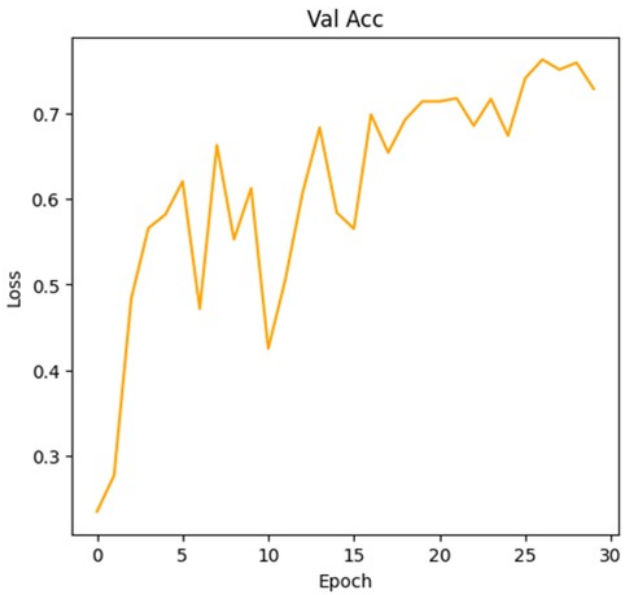
AI VIET NAM
@aivietnam.edu.vn

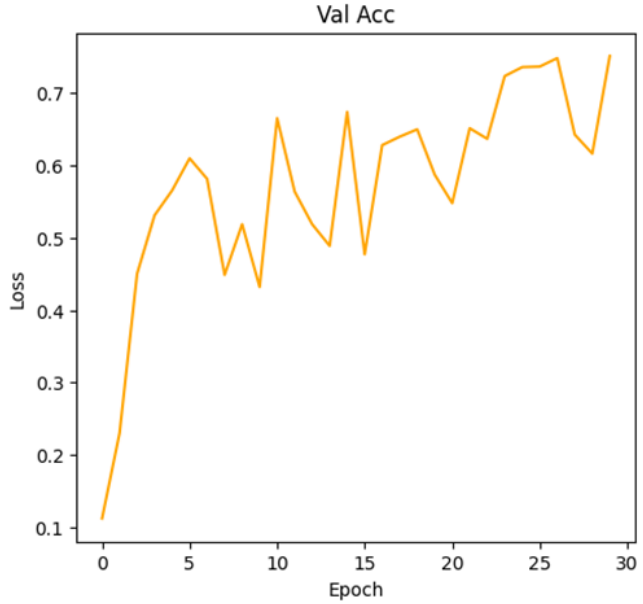
Response-based Knowledge

❖ Results

Student

Val accuracy: 0.7283
Test accuracy: 0.7278



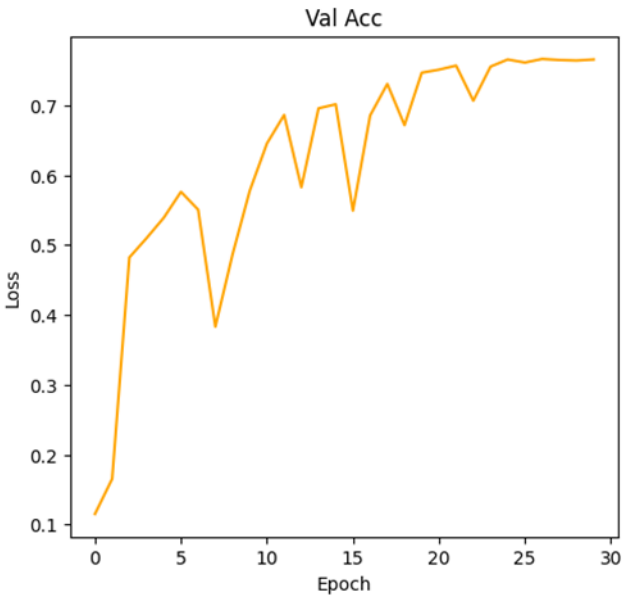


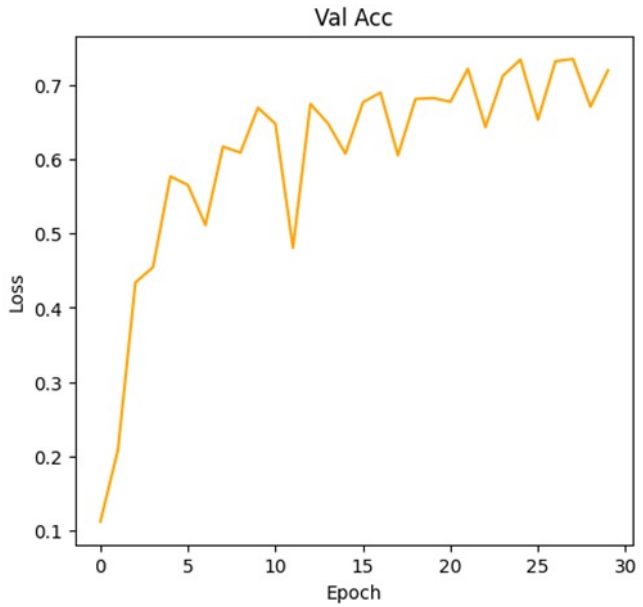
KD with Teacher pretrained on ImageNet

Val accuracy: 0.7509
Test accuracy: 0.7467

Teacher

Val accuracy: 0.7655
Test accuracy: 0.7540





KD with Teacher on the same dataset

Val accuracy: 0.7196
Test accuracy: 0.7351

AI VIET NAM
@aivietnam.edu.vn

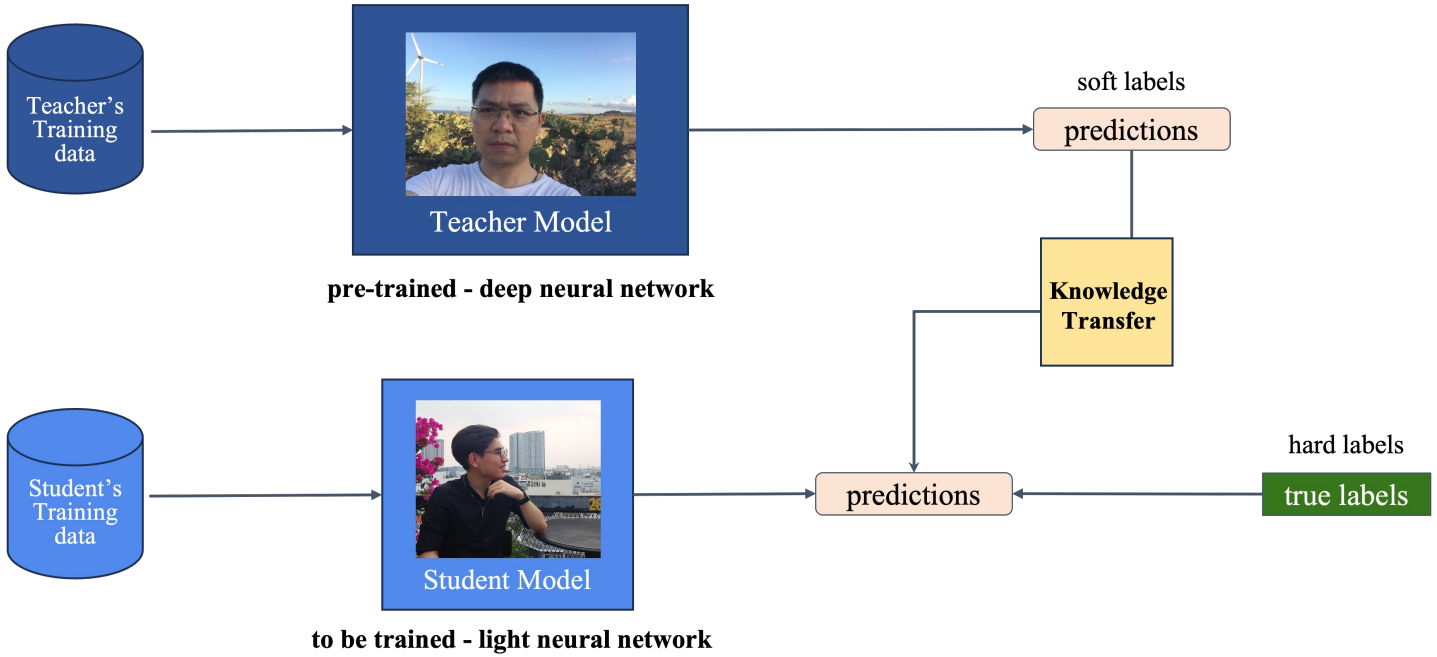
QUIZ

AI VIET NAM
@aivietnam.edu.vn

Summarization and Q&A

Summarization and Q&A

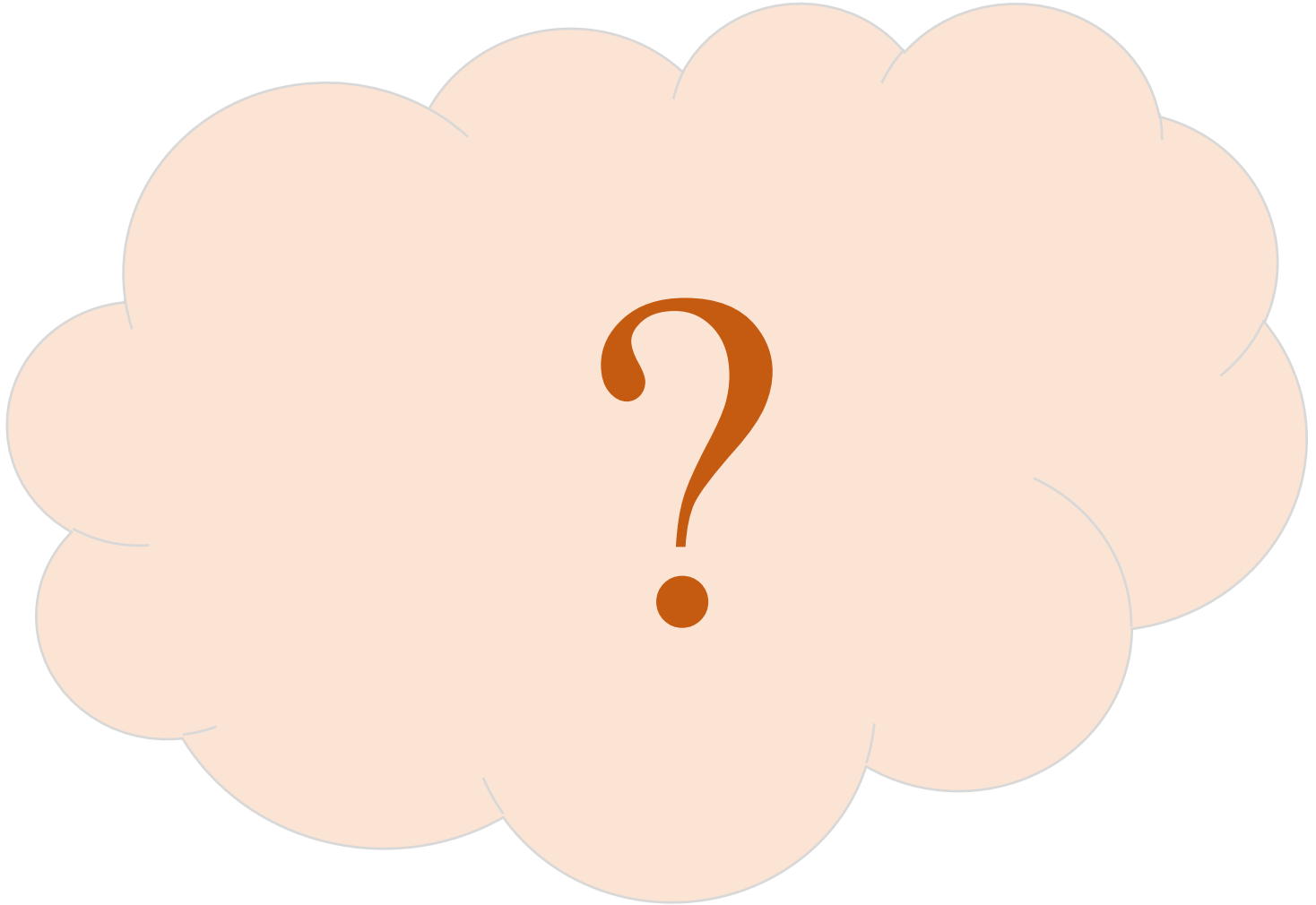
❖ Summarization



In this section, we have discussed about:

- Discuss about the motivation and definition of Knowledge Distillation technique.
- Some current challenges and applications of Knowledge Distillation.
- Basic approaches for Knowledge Distillation.
- Investigate how to implement the Response-based Knowledge Distillation method using PyTorch.

Question




Thank you!